

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belgaum -590014, Karnataka State, India



A PROJECT REPORT ON

“G-care: An AI Healthcare Assistant”

BACHELOR OF ENGINEERING

IN

DEPARTMENT

OF

COMPUTER SCIENCE ENGINEERING

[AI&ML]

Submitted by: -

Yeshrutha S

1EW23CI063

Abhay Vadagi

1EW23CI002



EAST WEST INSTITUTE OF TECHNOLOGY

Off Magadi Main Road, Bengaluru, Karnataka — 560 091

AY 2026-27

CERTIFICATE

This is to certify that the project report entitled G-Care: An AI Healthcare Assistant, submitted by Yeshrutha S and Abhay Vadagi, bearing USNs 1EW23CI063 and 1EW23CI002, to Visvesvaraya Technological University, Belagavi, in partial fulfilment of the requirements for the award of the Bachelor of Engineering degree in Computer Science and Engineering [AI&ML], is a record of project work carried out by them under my supervision. The contents of this report, in full or in part, have not been submitted to another institution or university for the award of a degree.

Signature:

Name of the Guide: Asst. Prof. Sreehari M.

Department: CSE [AI&ML]

Signature of HOD with seal

DECLARATION

We declare that this project report, G-Care: An AI Healthcare Assistant, submitted in partial fulfilment of the requirements for the Bachelor of Engineering degree in Computer Science and Engineering [AI&ML], is a record of original work carried out by us under the supervision of Sreehari M., Assistant Professor, CSE [AI&ML]. It has not formed the basis for the award of another degree at this or another institution or university.

We have acknowledged the work of others where it is cited. We distinguish our implementation, simulated software tests and reported physical demonstrations from capabilities that remain to be validated.

Signature of the students:

Name Signature

1. Yeshrutha S _____

2. Abhay Vadagi _____



EAST WEST INSTITUTE OF TECHNOLOGY

Off Magadi Main Road, Bengaluru, Karnataka - 560 091

Department of CSE [AI&ML]

CERTIFICATE

Certified that the project work entitled G-Care: An AI Healthcare Assistant was carried out by Yeshrutha S and Abhay Vadagi, USNs 1EW23CI063 and 1EW23CI002, students of East West Institute of Technology under the guidance of Asst. Prof. Sreehari M., in partial fulfilment of the requirements for the Bachelor of Engineering degree in Computer Science and Engineering [AI&ML] of Visvesvaraya Technological University, Belagavi, during the academic year 2026-27.

It is certified that corrections and suggestions indicated during internal assessment have been incorporated in the report deposited in the departmental library. The project report is submitted for approval in accordance with the academic requirements for project work.

Guide signature HOD signature Principal signature

External Viva

Sl. No. Name of examiner Signature with date

1. _____
2. _____

ABSTRACT

G-Care is a student prototype for coordinating remote elder care between doctors, caretakers and family guardians. It combines an ESP32-S3 breadboard prototype, a browser-based Watch Simulator and three role-specific portals. The hardware includes MAX30102, MAX30205 and MPU6050 modules and an OLED display. The prototype is not a finished wearable medical device, and no clinical or medical-grade accuracy claim is made.

The web application displays heart rate, oxygen saturation, temperature and other telemetry fields. Patient-specific simulation provides blood pressure, stress, hydration and breathing values for workflow testing; these fields are not measurements from the listed sensors. Rule-based anomaly detection creates alerts for abnormal readings. Stateful episode deduplication updates one record during a continuous abnormal condition. Recovery ends the condition episode but leaves an unacknowledged alert pending. A doctor can acknowledge the alert or schedule an appointment, which sends booking details to the care interfaces and the patient's watch.

The platform uses React 18, TypeScript, Tailwind CSS, Zustand and a Node.js HTTP backend. Signed JWTs and role checks control portal and record access. The audited local run uses JSON file storage; PostgreSQL support exists but was not validated against a live deployment. Medication entry, reminders and Taken actions were demonstrated, although complete historical adherence persistence remains a limitation.

The team reports physical movement-based fall triggering, simultaneous Watch/Guardian telemetry display, a sample blood-test PDF upload, Kannada voice interaction and a Hindi demonstration. Automated software tests verify simulated alerts, synchronization, JSON persistence and selected API workflows. English and Tamil are supported in code but are not claimed as fully demonstrated. Controlled human-fall trials, sensor accuracy benchmarking, clinical validation and measured latency remain future work.

Keywords: Elder Care; Telemetry Simulation; Rule-Based Alerts; Alert Episodes; Role-Based Access; Medication Reminders; Fall Detection; Multilingual Voice.

ACKNOWLEDGEMENT

We thank East West Institute of Technology for providing the facilities and academic support needed to carry out our project, G-Care: An AI Healthcare Assistant, as part of the Visvesvaraya Technological University requirements.

We thank Dr. Nagaraj G. Cholli, Principal of East West Institute of Technology, for supporting the project and making the institute's infrastructure available to us.

We express our gratitude to Dr. Venkatesha G., Head of the Department of CSE [AI&ML] and CSE [IoT, CyS and BCT], for the guidance and encouragement provided during the project.

We are grateful to our guide, Assistant Professor Sreehari M., and project coordinator Professor Shubha C. for their suggestions, reviews and support from the planning stage through implementation and report preparation. Their feedback helped us clarify the scope of the prototype and the work that still requires validation.

We also thank the teaching and non-teaching staff of the department, our classmates and everyone who supported the hardware demonstrations and software testing. This report distinguishes those demonstrations from automated tests and from future development goals.

YESHRUTHA S - 1EW23CI063

ABHAY VADAGI - 1EW23CI002

LIST OF FIGURES

Figure No.	Name	Page No.
1.1	High-Level Conceptual Architecture and Dataflow of G-Care	5
3.1	Multi-Tier Layered System Architecture of G-Care	11
3.2	DFD Level 0 - Context Level Diagram	14
3.3	DFD Level 1 - Detailed Functional Data Flow Diagram	15
5.1	End-to-End Operational Methodology Flowchart of G-Care	23
5.2	Shared I2C Bus Interfacing on Prototype Breadboard	25
5.3	Entity-Relationship Diagram of G-Care Platform	26
5.4	Use Case Diagram for the Tri-Tier G-Care System	27
7.1	Multi-Tier Testing Pipeline and Evidence Boundaries	38

LIST OF TABLES

Table No.	Name	Page No.
2.1	Qualitative Comparison of Monitoring Approaches and G-Care	8
4.1	Edge Hardware and Prototyping Components	16
4.2	Software Technology Stack	18
6.1	Breadboard Interfacing and Wiring Design	28
7.1	System Test Cases and Evidence-Based Verification Results	40

CONTENTS

Chapter / section	Name	Page No.
CHAPTER 1	INTRODUCTION	1
1.1	Motivation	2
1.2	Objectives	3
CHAPTER 2	LITERATURE REVIEW	6
2.1	Survey of Literature and Methodologies	6
2.2	Comparative Analysis	8
2.3	Research Gaps and Problem Formulation	9
CHAPTER 3	SYSTEM ARCHITECTURE	11
3.1	Architectural Framework	11
3.2	Subsystem Decomposition	12
3.3	Data Flow Modeling	14
CHAPTER 4	SYSTEM REQUIREMENTS	16
4.1	Hardware Requirements	16
4.2	Software Requirements	18
4.3	Functional / Non-Functional Requirements	20
CHAPTER 5	METHODOLOGY	22
5.1	Operational Flow	22
5.2	Detailed Design and Schema Modeling	24
CHAPTER 6	IMPLEMENTATION	28
6.1	Core Modules	28
6.2	Algorithms and Pseudocode	32
6.3	Security, Contracts and Storage	36
CHAPTER 7	TESTING	38
7.1	Methods and Evidence	38
7.2	Test Cases and Results	39
CHAPTER 8	CONCLUSION AND FUTURE SCOPE	42
	BIBLIOGRAPHY	45
APPENDIX A	SNAPSHOTS	46

Chapter 1

INTRODUCTION

Older adults living independently may need support with health observation, medication schedules and communication with family members or healthcare professionals. When caregivers live elsewhere or cannot remain present throughout the day, changes in a person's condition may not be noticed promptly. Remote monitoring tools can provide another source of information, but they do not replace clinical assessment or direct care.

Chronic conditions such as hypertension, diabetes and respiratory disease often involve repeated observations and medication routines. A dashboard that brings readings, reminders and care messages together can help users review this information. Its value depends on the quality of the measurements, correct patient identification, reliable state handling and appropriate follow-up. A prototype cannot establish improved medical outcomes simply by displaying these functions.

G-Care was developed to explore this coordination problem. It provides a Watch Simulator for the elder-facing interaction, a caretaker dashboard, a guardian portal and a doctor workspace. Users can view telemetry, receive abnormal-vital alerts, manage reminders and review appointment or medical-record information. The web interfaces share patient state so that a reading or alert can be reflected across the relevant views.

The project also includes a physical breadboard prototype using an ESP32-S3, MAX30102, MAX30205, MPU6050 and OLED. The team reports that the OLED displays sensor or simulated telemetry and that controlled movement or shaking can trigger the fall/emergency workflow. This demonstrates functional behavior on the prototype; it is not evidence of sensor accuracy, controlled human-fall detection or suitability for medical use.

1. Physical sensing node: The MAX30102 is used for optical heart-rate and SpO2 acquisition, the MAX30205 for temperature and the MPU6050 for motion. The OLED provides local feedback. Exact firmware settings and the deployed pin mapping were not available in the audited repository.

The physical setup is a development prototype rather than a wearable enclosure. Its results must be interpreted separately from browser-generated values, especially blood pressure, stress and hydration, which are not measured by the listed hardware.

2. Multi-portal web platform: The React application provides doctor, caretaker and guardian views, together with an interactive Watch Simulator. The simulator is used to inspect the elder-facing display and inject abnormal values while exercising alert and appointment workflows. It is a browser interface, not evidence that a complete wrist-worn device has been built.

3. Anomaly alerts and care actions: Rule-based checks classify selected readings as warning or critical. The episode manager keeps one record for a continuous elder-condition episode and updates it as readings change. The doctor can acknowledge a warning or book an appointment. Recovery does not automatically remove an alert that still needs acknowledgement.

1.1 Motivation

1. Support during periods without direct supervision: A family member or caretaker may not be able to observe an elder continuously. Sharing recent readings and event messages can help them decide when to check in. The project demonstrates notification handling rather than guaranteed emergency response or a measured clinical benefit.

2. Local feedback from a physical prototype: The breadboard and OLED let the team inspect sensor/display interaction independently of a web browser. This is useful for checking whether values and status messages can be rendered locally. Legibility and comfort during daily wearable use have not been evaluated.

3. Medication scheduling: Multiple medicines and different timings can be difficult to manage. G-Care offers entry fields, reminders and Taken actions to support this routine. A user acknowledgement records an interaction; it does not independently verify that a dose was swallowed.

4. Communication among care roles: The guardian, caretaker and doctor need different views of the same patient. The platform connects telemetry, alerts and booking information across those views. The prototype does not determine a diagnosis from those observations.

The motivation is therefore to build and evaluate a practical coordination workflow using available hardware and web tools. Claims about fewer hospital admissions, shorter emergency response times or reduced caregiver dependence would require a separate study.

5. Language and interaction: Small controls, dense text and unfamiliar terminology can make health interfaces difficult to use. G-Care provides localized messages and voice commands in English, Hindi, Kannada and Tamil. Kannada was functionally tested and Hindi was demonstrated; complete testing with older adults across all four languages remains to be done.

6. Availability of development components: The project uses commercially available sensor modules and web libraries. This makes a student prototype feasible, but no audited bill of materials or cost comparison supports a numerical claim of savings over commercial services. Deployment, maintenance, connectivity and enclosure costs also need to be considered.

1.2 Objectives

The aim is to implement a remote elder-care prototype that combines physical demonstrations, simulated telemetry and role-specific care workflows. The objectives below describe the project scope, not clinically validated outcomes.

- Physical hardware prototyping: Assemble an ESP32-S3 breadboard setup with MAX30102, MAX30205, MPU6050 and OLED modules. Demonstrate sensor/display operation and movement-triggered emergency behavior. Record firmware, wiring and raw measurements for later reproducibility.
- OLED display: Present changing heart-rate, oxygen and temperature values with a clear status area. Retain a rectangular display layout as a design choice; no measured refresh-rate or accessibility result is inferred from the illustrative code.
- Telemetry visualization: Share patient values between the Watch Simulator and role-specific dashboards without requiring manual page refresh. Separate measured device readings from simulation and manual entry, and show charts for review rather than diagnostic prediction.
- Patient-specific simulation: Keep distinct baselines for Usha, Lakshmi Devi and Venkatesh Rao. Use controlled abnormal values to exercise alert handling while avoiding persistent accidental abnormalities in the normal simulator baseline.

These objectives were evaluated through software tests and the functional demonstrations described in Chapter 7. The work does not include a current GPS module or controlled clinical evaluation.

- Abnormal-vital and emergency workflows: Evaluate the implemented heart-rate, SpO2, blood-pressure, breathing and stress rules. Route alerts to the care interfaces and watch. Treat thresholds as prototype configuration, not validated medical decision limits.
- Medication and routine reminders: Support medicine names, dose, instructions, frequency and timing, together with reminders and Taken actions. Distinguish persisted medicine definitions from incomplete historical adherence records.
- Doctor workspace: Provide patient views, alert acknowledgement, appointment booking, medical PDF uploads and clinical notes. A sample blood-test PDF was demonstrated; other document categories are interface options rather than fully validated ingestion pipelines.
- Multilingual interaction: Deliver reminder and appointment messages in the patient's preferred language. Support text, speech playback and voice recognition while acknowledging browser permissions, available voices and network dependence.
- Authentication and persistence: Use signed JWTs, server role checks, patient visibility rules and configured database adapters. JSON file storage is active in the audited local run; a PostgreSQL adapter is available but not established as a production deployment.

Architectural Description

1. Hardware and simulation layer: The breadboard provides local sensor/display behavior. Device endpoints accept telemetry, while the browser simulator generates patient-specific test values. Blood pressure, stress and hydration must not be presented as sensor measurements from this hardware set.
2. Processing and storage layer: A Node.js HTTP server handles routing, authentication and storage operations. Selected REST payloads use Zod validation. The browser anomaly detector and episode manager process simulated telemetry; the device route separately records readings and creates a fall alert when its flag is set.
3. Presentation layer: React interfaces show patient telemetry, notifications, reminders, appointment messages, medical documents and care information. BroadcastChannel/local browser events and periodic API reads support synchronization. The architecture does not use an implemented WebSocket/SSE stream or automated ambulance dispatch.

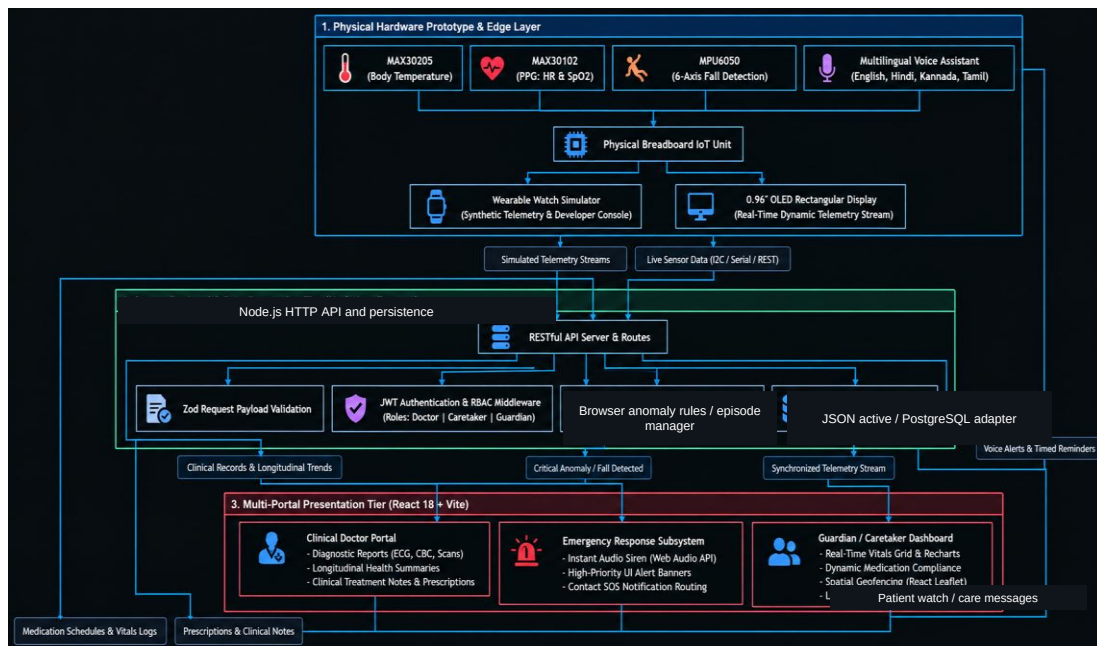


Figure 1.1: High-Level Conceptual Architecture and Dataflow of G-Care

The portals share selected patient information. Emergency messages are application alerts; automated calls, SMS and ambulance dispatch remain future work. This retained architecture figure has corrected labels for the current backend, browser episode processing and local JSON persistence. Its voice interface belongs to the browser Watch Simulator. It should not be interpreted as an embedded microphone or voice processor on the breadboard.

Chapter 2

LITERATURE REVIEW

The literature review covers remote patient monitoring, wearable sensing, fall detection, medication reminders and voice interfaces. These topics informed the project design. Findings reported by other researchers are not experimental results for G-Care, and a prototype feature does not establish a clinical benefit.

2.1 Survey of Existing Literature and Methodologies

The existing bibliography is retained with corrected citation numbering. References [1]-[6] cover the research background; [7] identifies the hardware documentation. The review does not infer that a component's published specification is an achieved system accuracy.

2.1.1 IoT-Based Remote Patient Monitoring Systems

Islam et al. [1] review IoT applications in healthcare, including connected sensors and information exchange. This provides background for separating sensing, communication, storage and presentation. For G-Care, those layers are used to organize a student prototype rather than to claim that interoperability or security concerns have been solved.

Majumder, Mondal and Deen [2] review wearable sensors for remote health monitoring. Their discussion motivates attention to sensing, communication, power and practical deployment. G-Care retains a breadboard implementation while using a browser simulator to test care workflows. Hardware miniaturization, battery life and long-duration operation still require separate engineering work.

The project uses role-specific web interfaces because different users need different information. The caretaker manages daily tasks, the guardian sees the assigned elder, and the doctor reviews alerts and clinical records. This is the project's design response to coordination needs; it is not a claim that all existing systems lack equivalent features.

The survey also motivates distinguishing data collection from decision support. G-Care's alerts are based on configured rules, and its recommendation text is not a trained diagnostic model. Measurement quality and patient context remain important before any care decision is made.

2.1.2 Photoplethysmography and Physiological Sensing

Allen [3] explains the principles and applications of photoplethysmography. The pulsatile component of an optical signal can support pulse estimation, while baseline variation and measurement conditions affect interpretation. This provides the background for using a MAX30102 module, but does not establish the accuracy of the assembled G-Care prototype.

The project therefore distinguishes raw optical sensing, estimated values and simulated display values. A numerical reading on the OLED or dashboard is not sufficient evidence of calibration. Comparison against reference instruments, signal-quality analysis and testing across motion and placement conditions remain future work.

2.1.3 Inertial Fall Detection and Emergency Alerts

Igual, Medrano and Plaza [4] review fall-detection approaches and the difficulties of comparing them under real-life conditions. Sensor placement, acquisition conditions, simulated falls and ordinary activity can affect evaluation. This is relevant to G-Care's MPU6050-based mechanism and its movement-triggered demonstration.

The team reports that physical breadboard movement produced a detection and emergency response. No controlled human-fall trial, sensitivity, specificity or false-positive/false-negative analysis was performed. The mechanism is therefore described as implemented and functionally demonstrated, with quantitative validation still required.

2.1.4 Medication Reminders and Senior Voice Interaction

Choudhry et al. [5] studied low-cost reminder devices in the REMIND randomized trial. The trial did not find improved adherence from the reminder devices alone. It cannot support the earlier report's claim that reminders necessarily reduce hospitalizations or improve compliance. G-Care evaluates reminder and acknowledgement functionality rather than treatment outcomes.

Nallam et al. [6] examine older adults' perceived benefits and barriers concerning voice assistants for health information. Participants raised access, privacy and trust concerns as well as potential benefits. The study motivates localized voice interaction, but does not justify a claim that G-Care removes digital-literacy barriers or has established high user acceptance.

2.2 Comparative Analysis of Healthcare Monitoring Systems

Table 2.1 is retained as a qualitative design comparison. Commercial product families and academic prototypes vary by version, region and configuration; the table is not a market benchmark, price survey or ranking of medical performance. Broad statements such as universal absence, high accuracy or a fixed commercial cost have been replaced with qualifications.

Table 2.1: Qualitative Comparison of Monitoring Approaches and G-Care

The G-Care column distinguishes implemented workflow features from simulated metrics and future hardware. Presence of a web feature does not mean that the corresponding physiological measurement or clinical outcome has been validated.

Feature / capability	Commercial smartwatches	Clinical RPM products	Hospital portals	Academic prototypes	G-Care prototype
Vitals	Varies by model; not all BP/SpO2 functions available	Sensor/service dependent	Measured or entered records	Varies by sensors	HR/SpO2/temp hardware; BP/stress/hydration simulated
Anomaly alerts	Vendor-specific behavior	Service-specific escalation	Varies by deployment	Often threshold or model based	Configured rules plus stateful episodes; simulated tests
Fall/location	Capabilities depend on device	Device/service dependent	Not implied by portal	Hardware and evaluation vary	MPU6050 movement demo; location simulated; no current GPS
Care roles	Account/sharing options vary	Provider/caregiver access varies	Deployment-specific access	Project-dependent roles	Doctor, caretaker, guardian roles and patient visibility

Table 2.1 (continued): Qualitative Comparison of Monitoring Approaches and G-Care

2.3 Research Gaps and Problem Formulation

1. Coordination among roles: Patient readings, care tasks and clinical decisions may be distributed across different tools. The design goal is to make selected information available in role-appropriate views. This does not imply that every existing product omits caregivers or that G-Care implements hospital-level interoperability.

2. Telemetry and medication context: Presenting reminders alongside readings can help users review daily activity. A Taken action represents an acknowledgement, and it must not be treated as verified drug ingestion or a complete longitudinal adherence record. Reliable history and restart behavior still require further validation.

Feature / capability	Commercial smartwatches	Clinical RPM products	Hospital portals	Academic prototypes	G-Care prototype
Medication reminders	Available features vary	Service dependent	Prescription workflows vary	Implementation dependent	Entry/reminder/Taken demo; history persistence incomplete
Medical records	Integration dependent	Integration dependent	Common EHR function; varies	Project dependent	Sample blood-test PDF demonstrated; JSON record tests
Voice / languages	Vendor/region dependent	Service dependent	Accessibility varies	Project dependent	EN/HI/KN/TA supported; KN tested, HI demonstrated
Deployment / cost	Model/service dependent	Purchase/subscription dependent	Hosting/licensing dependent	Scope dependent	Web/broadboard prototype; no verified total cost comparison

3. Alert handling: Repeated abnormal readings can create unnecessary records if each telemetry tick is treated as a separate event. G-Care addresses this software issue with an elder-plus-condition episode identity. This is different from measuring the physiological false-alarm rate, which depends on sensor quality and real-world testing.
4. Language and interface needs: Regional-language messages may help some users interact with reminders and appointment information. Kannada interaction and a Hindi demonstration provide functional evidence. Broader language testing, older-adult usability sessions and accessibility measurements are still needed.
5. Deployment and ownership of records: A web application can make information available to multiple roles, but its data must remain consistent across refresh, disconnection and server restart. The local JSON backend and browser stores support selected records; production database operation, medication adherence history and wider device integration remain areas for improvement.

Problem Formulation

The project problem is to build an elder-care coordination prototype in which users can view telemetry, receive rule-based alerts, manage reminders and communicate appointment or medical-record information through separate doctor, caretaker and guardian interfaces.

G-Care combines a physical breadboard demonstration with a browser Watch Simulator. The simulator allows repeatable abnormal-vital scenarios without presenting those values as patient measurements. The application should retain one logical alert during an ongoing condition, preserve it until a user action resolves it, and allow a later condition episode after recovery.

The resulting evaluation concerns implementation and workflow behavior. It does not establish clinical efficacy, cost savings, medical-device readiness or a reduction in caregiver workload. Those questions require a planned study, suitable reference instruments and a documented deployment environment.

Chapter 3

SYSTEM ARCHITECTURE

G-Care separates sensing/simulation, communication, application logic, storage and presentation. The audited application is a React single-page frontend with a Node.js HTTP backend. It can display device-origin readings and uses patient-specific simulation when fresh device data is not selected.

The current local run uses data/db.json. PostgreSQL is an alternative adapter selected by configuration, not evidence of a deployed production database. Browser state and API data must be distinguished, especially for reminders and telemetry history.

3.1 Architectural Framework and Layered Organization

The five conceptual layers are retained in Figure 3.1. Labels have been corrected to remove current GPS hardware and unsupported transport/backend claims. The current UI uses browser synchronization and an approximately three-second polling cycle, rather than an implemented WebSocket or SSE feed. This configured interval is not a measured end-to-end latency.

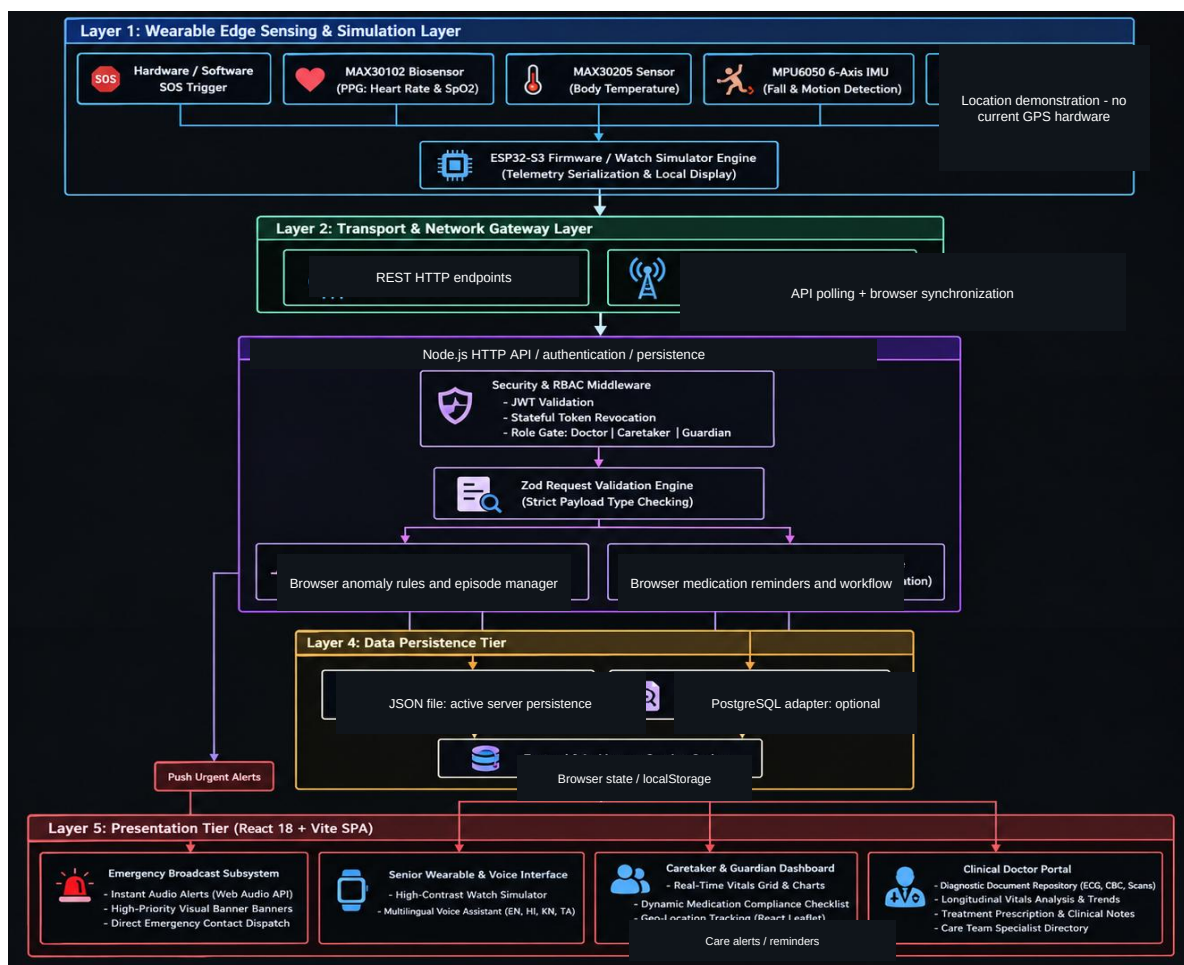


Figure 3.1: Multi-Tier Layered System Architecture of G-Care

3.2 Subsystem and Module Decomposition

3.2.1 Physical Breadboard Sensing Subsystem

The team identifies an ESP32-S3 breadboard with MAX30102, MAX30205, MPU6050 and OLED modules. The MAX30102 provides red/infrared optical sensing for heart-rate and SpO2 estimation; the MAX30205 supplies temperature data; the MPU6050 provides acceleration and angular-rate information. The OLED displays changing values or status messages.

The repository does not contain the deployed firmware or raw hardware logs. Sensor operating ranges, sample rates, calibration, bus frequency and exact display timing are not independently verified results. Published component specifications [7] must not be substituted for measured prototype accuracy.

3.2.2 Anomaly Detection and Alert Episodes

The browser detector implements the following prototype rules:

- Heart rate: warning above 100 or below 55 bpm; critical above 120 or below 48 bpm.
- SpO2: warning below 93%; critical below 90%. No oxygen warning is created at 93-94.9% by this detector, although recovery uses at least 95%.
- Blood pressure: warning at systolic ≥ 140 or diastolic ≥ 90 mmHg; critical at ≥ 160 or ≥ 100 . Low BP is warning below 90/60 and critical below 85/55, using either component.
- Breathing/stress: the detector also evaluates breathing rate above 26/min and stress above 85/100.

A stable elder ID and canonical condition name identify an episode. Repeated readings update the same record. Acknowledgement silences that episode without changing its measurements; recovery rearms detection but does not resolve an unacknowledged record. The database serializes JSON creation and uses an elder-row lock in the PostgreSQL path.

3.2.3 Medication and Routine Scheduling

Medicine definitions contain name, dose, timing, instructions and frequency. Reminders are evaluated in the browser. Users can mark a dose or reminder Taken; complete historical adherence persistence is not established by the current tests.

3.2.4 Caretaker and Guardian Dashboards

The caretaker dashboard combines patient information, vitals, medications and alarms. The guardian portal restricts patient visibility and shows telemetry, alerts and care information. Recharts displays configured history or generated series; chart presence alone does not demonstrate weeks of measured patient data.

React Leaflet supports location/safe-zone visualization. Current coordinates and geofence demonstrations are software data, not readings from a GPS module on the breadboard. A current satellite-tracking claim is therefore excluded from the hardware scope.

3.2.5 Doctor Portal and Medical Workspace

The doctor portal supports alert acknowledgement, appointment scheduling, medical PDF upload, document viewing and clinical notes. The team demonstrated a sample blood-test PDF with patient, document category, doctor, date and View Details/Open PDF information. Guardian and caretaker access was checked.

The backend stores document metadata and base64 PDF data in the selected data layer. Validation uses PDF/content checks and clinical keyword heuristics; it does not verify diagnostic correctness, authenticity or every ECG, scan and prescription format. A note editor is implemented, but it is not a complete electronic health record system.

3.2.6 Multilingual Voice Subsystem

The Watch Simulator supports English, Hindi, Kannada and Tamil message generation. It uses browser speech recognition and a server TTS endpoint with browser speech-synthesis fallback. Kannada was tested and Hindi demonstrated; English/Tamil voice quality and recognition accuracy are not established by those demonstrations.

Appointment messages include date, time, doctor and a preparation reminder. Speech completion closes the watch confirmation; interrupted playback does not count as completion. The manual SOS control is below the initial watch content and requires scrolling.

3.2.7 Authentication and Role Separation

The backend signs HMAC-SHA256 JWTs and checks the authenticated role and patient access. ProtectedRoute also gates frontend routes. Doctor, caretaker and guardian separation was demonstrated, but this is not a penetration-test result or a certification of the entire application.

The authentication code stores revoked token identifiers and checks them when authenticating requests. The actual logout path is `/api/auth/logout`. Old-token rejection after logout exists as an implementation mechanism; it is not reported here as an experimentally verified test because no matching test evidence was supplied.

Selected REST handlers validate request bodies with Zod. This should not be generalized to every route: the device telemetry handler manually parses fields and supplies defaults.

3.3 Data Flow and System Interaction Modeling

3.3.1 Level 0: Context Data Flow Diagram

Figure 3.2 retains the actors and conceptual exchanges. The external-emergency actor is a design boundary; current SOS notifications are in-application events, not automatically delivered ambulance calls.

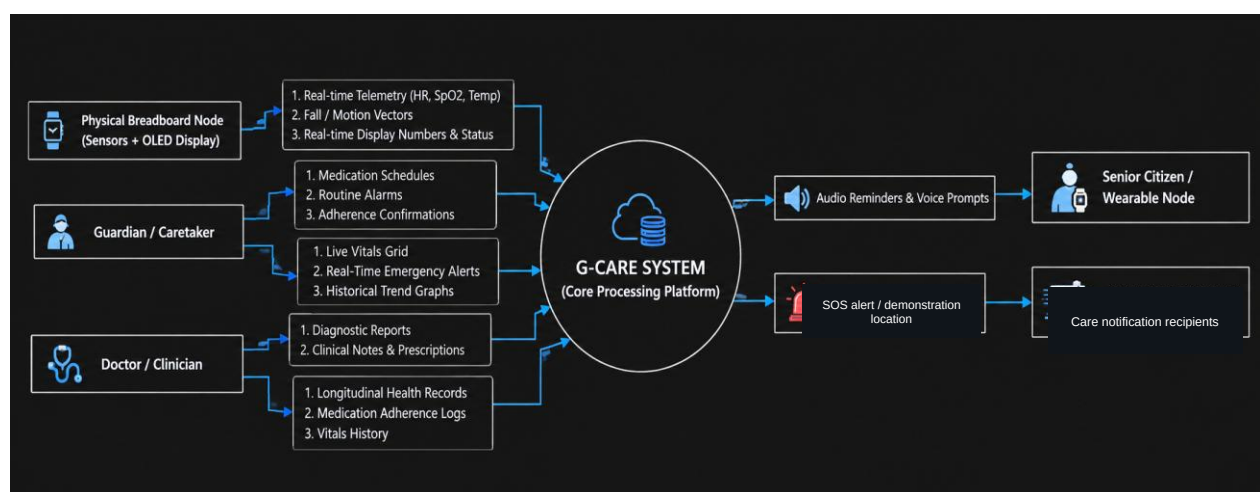


Figure 3.2: DFD Level 0 - Context Level Diagram

3.3.2 Level 1: Functional Data Flow Diagram

Figure 3.3 separates telemetry, alert episodes, reminders, clinical records and storage. These are conceptual processes; the final browser/backend allocation and evidence limitations are described in Sections 3.2 and 6.3.

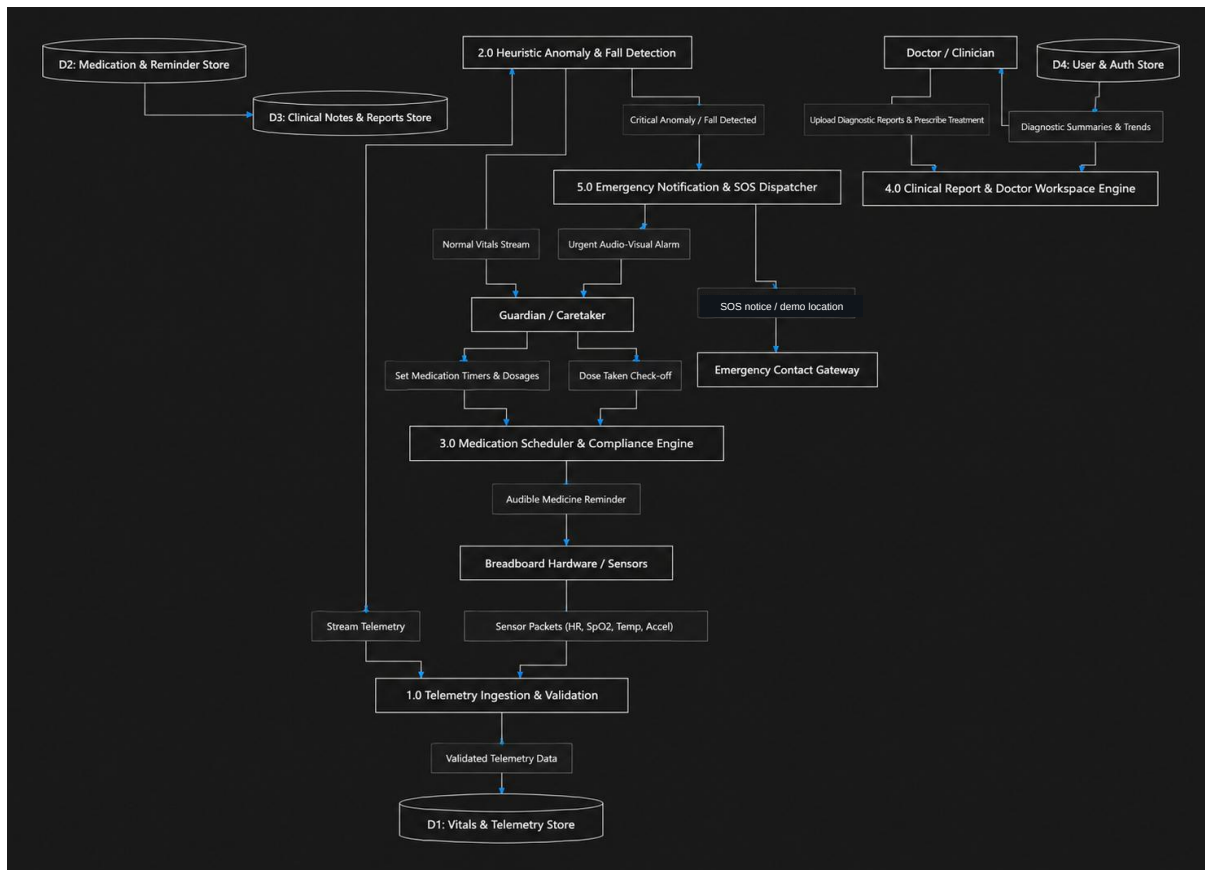


Figure 3.3: DFD Level 1 - Detailed Functional Data Flow Diagram

The retained DFD is a logical model. In the implemented web workflow, anomaly evaluation and episode handling run in the browser, with alert persistence handled by the backend. Hardware fall flags are handled separately by the device endpoint. Medication reminder completion is not a validated long-term clinical adherence record.

Chapter 4

SYSTEM REQUIREMENTS

The requirements define the prototype's hardware, software and care workflows. They distinguish functions currently available from design goals that still need validation. Requirements for medical accuracy, quantified latency, uptime and accessibility are targets rather than achieved results.

4.1 Hardware Requirements

4.1.1 Physical Breadboard Prototype Specifications

The physical setup reported by the team includes the ESP32-S3, MAX30102, MAX30205, MPU6050 and OLED. Table 4.1 preserves the component details while distinguishing reported hardware from unverified wiring, timing and power options. The current prototype contains no GPS receiver.

Table 4.1: Edge Hardware and Prototyping Components

Component specifications describe the modules, not validated physiological accuracy of the assembled prototype. Breadboard operation is not equivalent to a safe, calibrated wearable.

Component	Model / design specification	Function / evidence boundary
Prototyping surface	Solderless breadboard and jumper wires; original design: 400 tie-points	Physical prototype reported; exact board size not independently logged.
Microcontroller	ESP32-S3; original design names WROOM-1, dual-core LX7 up to 240 MHz, 512 KB SRAM, Wi-Fi/BLE	Sensor/display controller [7]. Board variant, firmware and actual GPIO mapping need recording.
Optical sensor	MAX30102 red/infrared PPG module	HR/SpO2 estimation; original 40-180 bpm and 70-100% figures are not validated measurement ranges.
Temperature	MAX30205 temperature module	Temperature sensing; published component accuracy is not measured breadboard accuracy.

Table 4.1 (continued): Edge Hardware and Prototyping Components

4.1.2 Development, Server and Client Workstations

The existing planning recommendations are retained: an Intel Core i5, AMD Ryzen 5 or comparable processor; 8 GB RAM with 16 GB preferred for concurrent development; approximately 20 GB of available storage for tools and dependencies; and a network connection for API/TTS access. These are suggested development resources, not experimentally determined minimum system requirements.

A desktop or laptop with a modern browser is used for development and demonstrations. Responsive layouts also target tablets and smartphones. The original suggested desktop resolution of 1366 x 768, with 1080p preferred, is a viewing recommendation rather than a tested compatibility guarantee.

OLED viewing is local to the prototype. The browser Watch Simulator is a separate interface. Audio playback, speech recognition and vibration depend on browser support, permission, device capability and available services.

The audited local web server uses HTTP. HTTPS termination, host capacity and secure production deployment must be configured and tested before external use. No throughput benchmark or required network bandwidth is inferred from the local demonstration.

Component	Model / design specification	Function / evidence boundary
Motion sensor	MPU6050 accelerometer/gyroscope, six axes	Physical movement-triggered fall demo reported; 50 Hz and >3 g are unverified design parameters.
Display	Original design: 0.96-inch, 128 x 64 OLED with SSD1306/I2C	OLED displays sensor/simulated telemetry; refresh rate not measured.
Pull-ups	Original design: 4.7 kilohm resistors	I2C requirement depends on breakout pull-ups and bus. SDA/SCL actual GPIOs need confirmation.
Power	USB-powered prototype; original alternative: 3.7 V, 800 mAh Li-Po plus regulation	Battery specification and endurance not validated. Use module-appropriate regulated supply and common ground.

4.2 Software Requirements

The web stack below is verified from package.json and the source. Dependency versions are declared requirements, not evidence that every platform or tool combination has been tested. Firmware tooling is retained as a design requirement because the deployed embedded source was not found in this workspace.

Table 4.2: Software Technology Stack

React components, TypeScript models, Zustand stores and Recharts provide the main presentation/state functions. Vite builds the client. Tailwind CSS and shadcn/ui/Radix supply styling and controls. The backend is written in JavaScript and uses Node's native HTTP module, so type safety is not claimed across every server route.

Domain	Technology / requirement	Version / scope	Role / qualification
OS	Windows; Linux/macOS possible	Local Windows run	Other OS compatibility not fully tested
Firmware IDE	Arduino IDE / PlatformIO	Original Arduino 2.3+ proposal	Firmware/tool versions not archived
Display / bus	Adafruit SSD1306/GFX, Wire	Illustrative C++ example	Deployed library versions not verified
Frontend	React	18.3.1 declared	Role-specific components
Language	TypeScript	5.8.3 declared	Frontend models; server is JavaScript
Build	Vite	8.1.5 declared	Production build verified
Styling	Tailwind CSS / shadcn-ui	3.4.17 / Radix components	Responsive layouts; no WCAG audit
State	Zustand	5.0.12 declared	Application/auth/guardian stores
Charts	Recharts	2.15.4 declared	Charts may use simulated history

Table 4.2 (continued): Software Technology Stack

The code supports alternative JSON and PostgreSQL storage paths. JSON is selected when DATABASE_URL is absent; a database connection failure is not documented as automatic, lossless migration to JSON. The local demonstration and isolated persistence tests use the JSON adapter.

The repository contains Playwright configuration, but the recorded verification run is Vitest plus the production build. Browser tests rendered with React/jsdom and mocked services must not be described as full cross-browser or live speech-recognition experiments.

Domain	Technology / requirement	Version / scope	Role / qualification
Mapping	React Leaflet / Leaflet	4.2.1 / 1.9.4	Software location/safe-zone views; no GPS hardware
Voice / audio	Web Speech, TTS endpoint, WAV assets	Browser/service dependent	Four languages supported; KN/HI demonstration boundary
Runtime	Node.js	Engine ≥ 20	Native HTTP server; no Express implementation
Validation	Zod	3.25.76 declared	Selected REST schemas; device parser differs
Authentication	Node crypto	HS256 JWT; PBKDF2-SHA512	Custom active implementation; not bcrypt/jsonwebtoken path
Storage	JSON file / PostgreSQL adapter	Local JSON active; pg 8.11.3 declared	Live PostgreSQL operation not verified
Tests	Vitest / jsdom; Playwright config	Vitest 3.2.x; 117 tests run	Simulated/mockd and isolated JSON/HTTP checks; no complete browser matrix

4.3 Functional and Non-Functional Requirements

4.3.1 Functional Requirements

- FR1: Hardware acquisition and local display. Use the breadboard sensors and OLED to demonstrate changing values and a movement-triggered status response. Keep sensor-origin readings separate from substituted/simulated telemetry. Record the final firmware and actual pin mapping before claiming reproducible acquisition settings.
- FR2: Authentication and role separation. Authenticate doctor, caretaker and guardian users, issue signed tokens, gate role-specific routes and enforce patient access on the server. A revocation implementation exists, but logout-replay behavior needs an explicit experiment before being marked verified.
- FR3: Telemetry monitoring. Display patient values on the watch and dashboards and update them without manual refresh. Heart rate, SpO2 and temperature may come from device telemetry; BP, stress, hydration and other unsupported sensor metrics are simulated or manually supplied in this prototype.
- FR4: Abnormal-vital and emergency alerts. Apply the configured warning/critical rules, maintain continuous condition episodes, update repeated readings in place and keep each alert pending until a manual care action. Support software SOS and device fall flags without claiming validated physical geofencing.
- FR5: Medication and reminders. Allow medication entry, dose/timing instructions and reminders. Provide Taken actions and care notifications. Persistent medicine definitions exist, but complete adherence history across refresh/restart and alert workflows remains a validation gap.
- FR6: Doctor workspace. Provide patient trends, alert review, appointment booking, medical PDF upload/viewing and clinical notes. Accepting a category label is not proof that all clinical document formats have been tested.

The role flows and patient-specific visibility are evaluated as application behavior. No diagnosis, dose recommendation or automatic external emergency service response is established by these requirements.

- FR7: Multilingual voice interaction. Generate reminder and appointment messages in English, Hindi, Kannada and Tamil, with speech playback and supported commands. Kannada and Hindi have demonstration evidence; wider language validation is required. An appointment announcement should finish before its watch confirmation closes.

4.3.2 Non-Functional Requirements

- NFR1: Performance. The original design proposed an OLED refresh near 2 Hz, dashboard propagation below 500 ms and anomaly evaluation below 50 ms. No corresponding measurement logs exist, so these are unverified targets. The current browser polling period is 3 seconds and must not be represented as a 500 ms latency measurement.
- NFR2: Reliability. A 99.9% uptime figure is not supported by an availability study. The implementation should be evaluated for server restart, connection loss and storage failures. JSON and PostgreSQL are alternative configured adapters, not a demonstrated automatic failover pair.
- NFR3: Usability/accessibility. Clear text, contrast and usable controls are design goals. The original WCAG 2.1 AA, 4.5:1 contrast, 16-18 px text and 48 x 48 px touch-target values remain evaluation criteria rather than a completed conformance audit.
- NFR4: Security. Protect deployed traffic with HTTPS/TLS, configure secrets and limit access. The source hashes passwords using salted PBKDF2-SHA512 with 120,000 iterations, not bcrypt. Selected payloads use Zod, but the device route needs separate input-validation review. These controls do not constitute a security certification.
- NFR5: Portability. Responsive desktop/mobile interfaces are implemented. Chrome, Firefox, Edge and Safari require browser-specific testing, especially for speech and background behavior; uniform compatibility is not claimed.

These requirements identify work to be measured or reviewed. They are not substituted for observed PASS/FAIL results in Chapter 7.

Chapter 5

METHODOLOGY

This chapter describes the prototype's operating flow, threshold rules, alert episode lifecycle, reminders and design models. The hardware demonstration and the browser simulation are separate evidence sources. The physical fall mechanism is reported by the team; its exact firmware was not present for code-level verification.

5.1 Proposed Methodology and Operational Flow

The workflow has five stages: acquire or generate telemetry; evaluate applicable rules; create/update an episode alert; persist and synchronize selected state; and present care actions, reminders or localized messages. Acknowledgement and booking are user actions rather than consequences of numerical recovery.

5.1.1 Telemetry Acquisition and Signal Processing

The physical prototype uses optical, temperature and inertial modules. The original report specified 50 Hz inertial sampling, 10 Hz optical sampling and a 16-sample moving average. Without deployed firmware or measurements, those settings are retained only as proposed acquisition/filtering parameters. No claim is made that filtering eliminates motion artifacts or that temperature was calibrated to a demonstrated accuracy.

The web model includes elder ID, timestamp, heart_rate, spo2, systolic_bp, diastolic_bp, skin_temp, stress, hydration and breathing_rate, with fall/panic/shiver flags. The current device handler stores a summarized reading; it does not ingest the complete raw acceleration/gyroscope vectors described in the original conceptual frame.

The browser cycle checks recent device telemetry for elder-1. Otherwise it generates bounded, mean-reverting values using patient-specific baselines. That behavior supports repeatable workflow testing but can substitute simulation after a device read, so a displayed number is not necessarily a continuously measured sensor stream. Figure 5.1 retains the conceptual flow with labels identifying design-only persistence and notification steps.

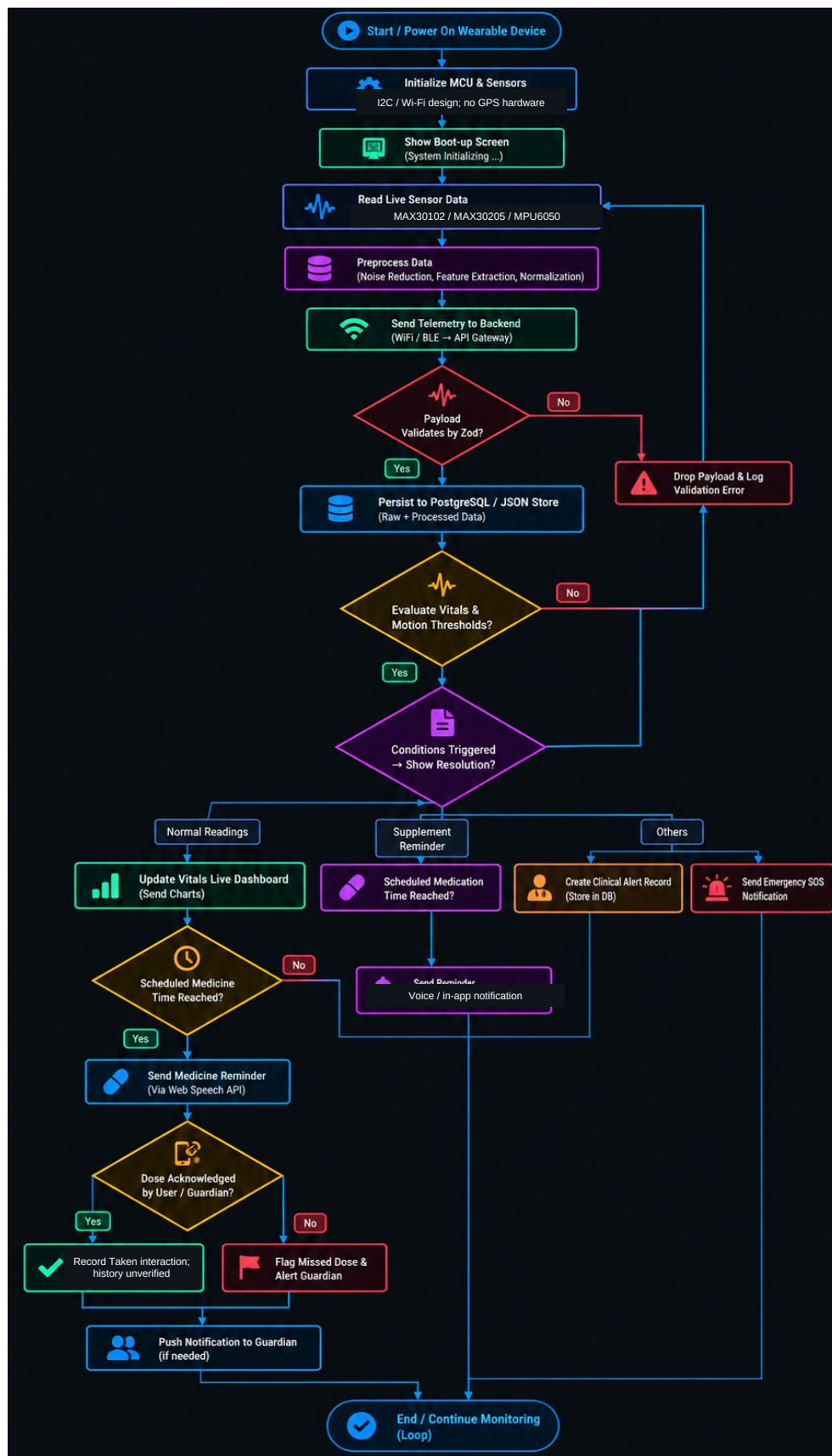


Figure 5.1: End-to-End Operational Methodology Flowchart of G-Care

Design flow; clinical validation and continuous adherence history are not established.

5.1.2 Threshold Rules and Episode Lifecycle

The implemented rules in `anomalyDetector.ts` classify synthetic/entered values, rather than diagnose a medical condition.

Heart rate: critical when $HR > 120$ bpm or $HR < 48$ bpm; warning for $100 < HR \leq 120$ or $48 \leq HR < 55$; otherwise no heart-rate anomaly. Recovery is stricter: $60 \leq HR \leq 95$.

SpO2: critical below 90%; warning for $90 \leq SpO_2 < 93\%$. At 93% or higher no new oxygen anomaly is returned. A low-oxygen episode is marked recovered at $\geq 95\%$, providing hysteresis between warning and recovery.

Blood pressure: high warning at systolic ≥ 140 or diastolic ≥ 90 , high critical at ≥ 160 or ≥ 100 . Low warning occurs below 90/60 and low critical below 85/55, using either component. Recovery requires systolic 96-134 and diastolic 65-84. Breathing recovery is 12-22/min and stress recovery is $\leq 65/100$.

The physical fall design uses acceleration magnitude $\sqrt{A_x^2 + A_y^2 + A_z^2}$, with candidate free-fall, impact and post-impact stages. The original 0.5 g, 3.0 g, 60-degree and five-second values are illustrative design parameters, not verified deployed firmware settings or clinical performance results.

Episode deduplication uses `elderId` + canonical anomaly type, such as `elder-3:LOW_SPO2`. Changing values, timestamps and generated IDs are excluded from the key. Repeated abnormal values update one current record, including severity changes. Acknowledgement latches that episode until recovery. Recovery ends the physiological episode without resolving a pending alert; a later abnormal condition may create a new record.

5.2 Detailed System Design and Schema Modeling

5.2.1 Shared I2C Bus and Hardware Interfacing

The original bus diagram is retained with unverified numeric GPIO labels replaced by named SDA/SCL connections. GPIO22 is not an ESP32-S3 GPIO [7]; the actual deployed mapping must be documented rather than guessed.

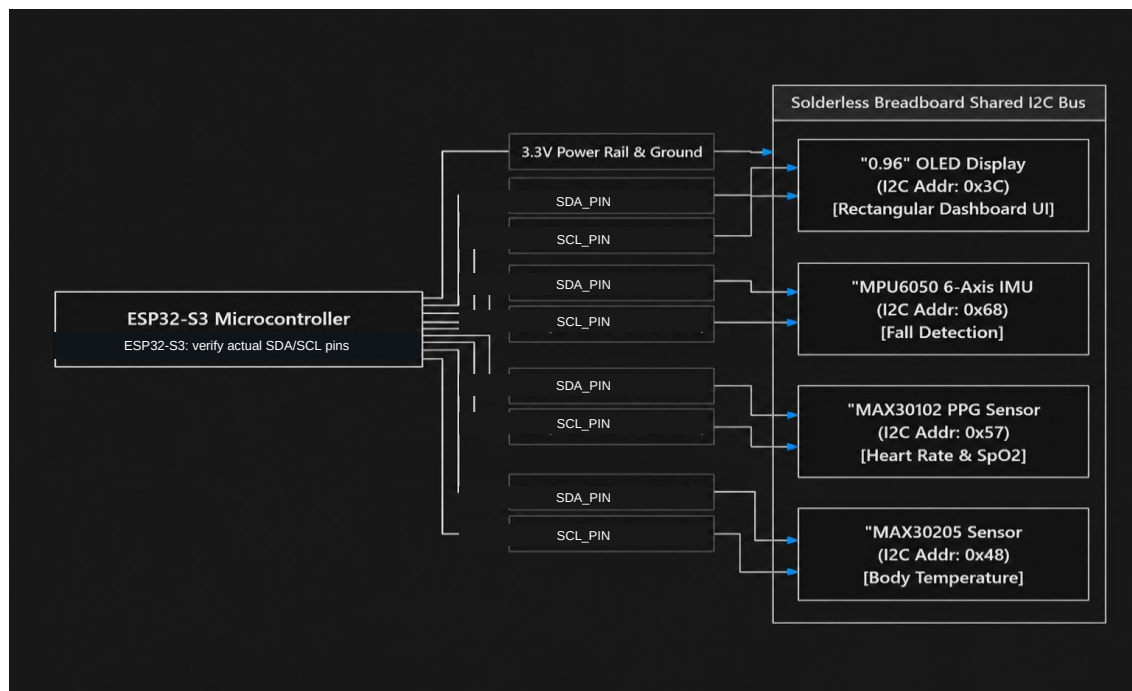


Figure 5.2: Shared I2C Bus Interfacing on Prototype Breadboard

I2C Bus Addressing

The original design lists OLED/SSD1306 at 0x3C, MAX30102 at 0x57, MAX30205 at 0x48 and MPU6050 at 0x68. These are design/configuration values rather than independently verified bus-scan results. Actual module addressing and address-pin configuration should be checked against the hardware.

The original 400 kHz bus setting is retained as a proposed configuration, not a measured operating result. All modules need a common ground, suitable power and correct breakout-board electrical requirements. Existing pull-ups must be considered before adding external resistors.

5.2.2 OLED Rectangular UI Layout

The design uses a 128 x 64 OLED with a rectangular boundary from (0,0) to (127,63). A title area spans approximately rows 0-14, followed by the vitals rows and a lower status region. The layout is illustrative; no senior-user readability test has been recorded.

Heart rate is shown in bpm and oxygen saturation as a percentage. The example values HR 76 and SpO2 98 are display examples, not experimental measurements. Firmware should distinguish unavailable/simulated values from sensor-origin readings.

The temperature row displays degrees Celsius; an example such as 36.8 C illustrates formatting rather than calibration. The lower region, approximately rows 51-63, can display normal status or a fall-detection message.

5.2.3 Entity-Relationship Schema Modeling

Figure 5.3 retains the relational design for users, elders, medications, alarms, vitals, clinical notes, reports and alerts. PostgreSQL tables exist in the adapter; active local records are stored in JSON. The JSON file does not inherit relational constraints or ACID guarantees merely because an ER model is present.

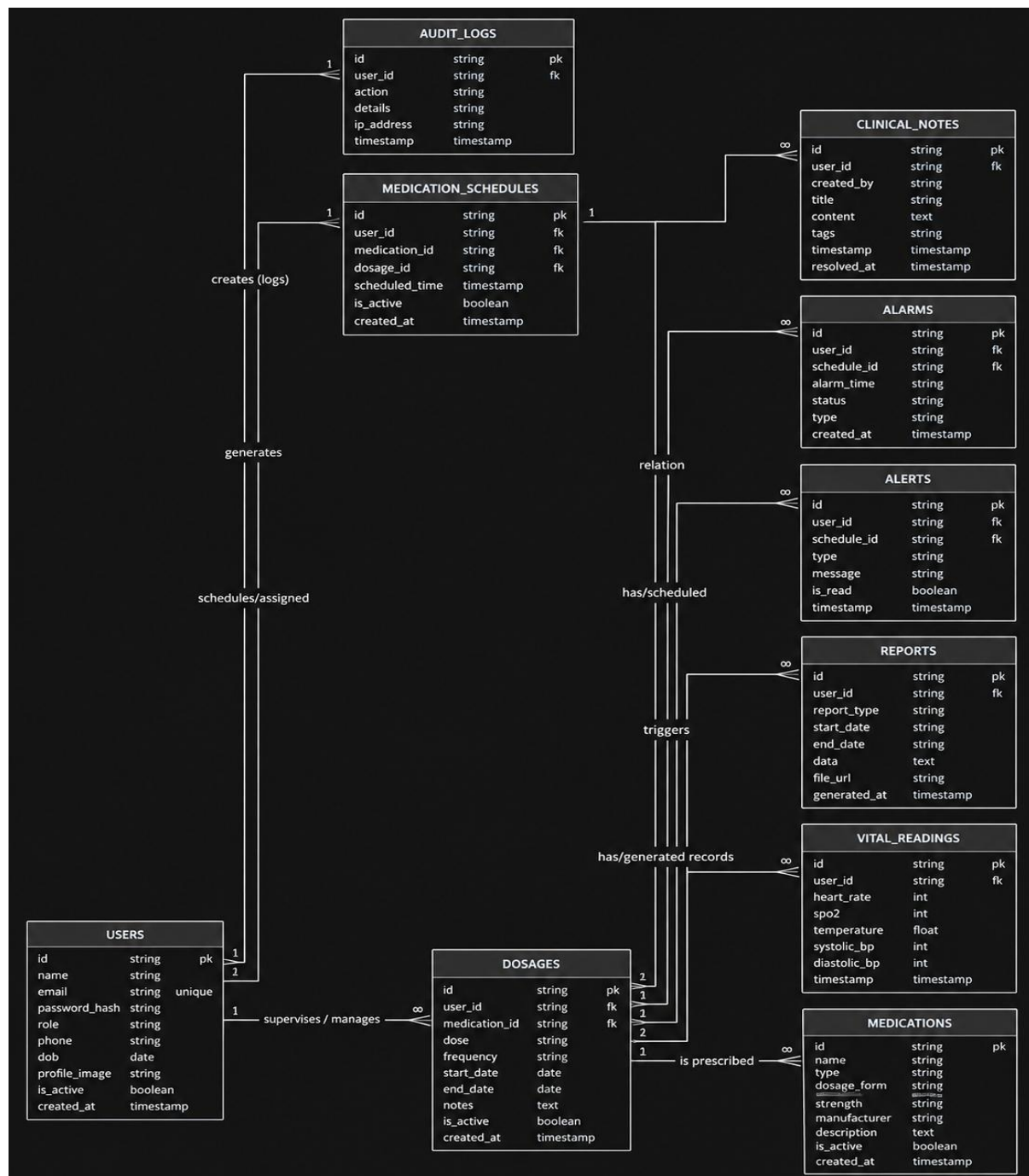


Figure 5.3: Entity-Relationship Diagram of G-Care Platform

5.2.4 Use Case Modeling

The use-case diagram retains the elder interface, family guardian/caretaker and doctor roles. It models reviewing readings, managing reminders, acknowledging alerts and exchanging clinical information. It is a design model, not proof that every interaction has been physically or experimentally validated.

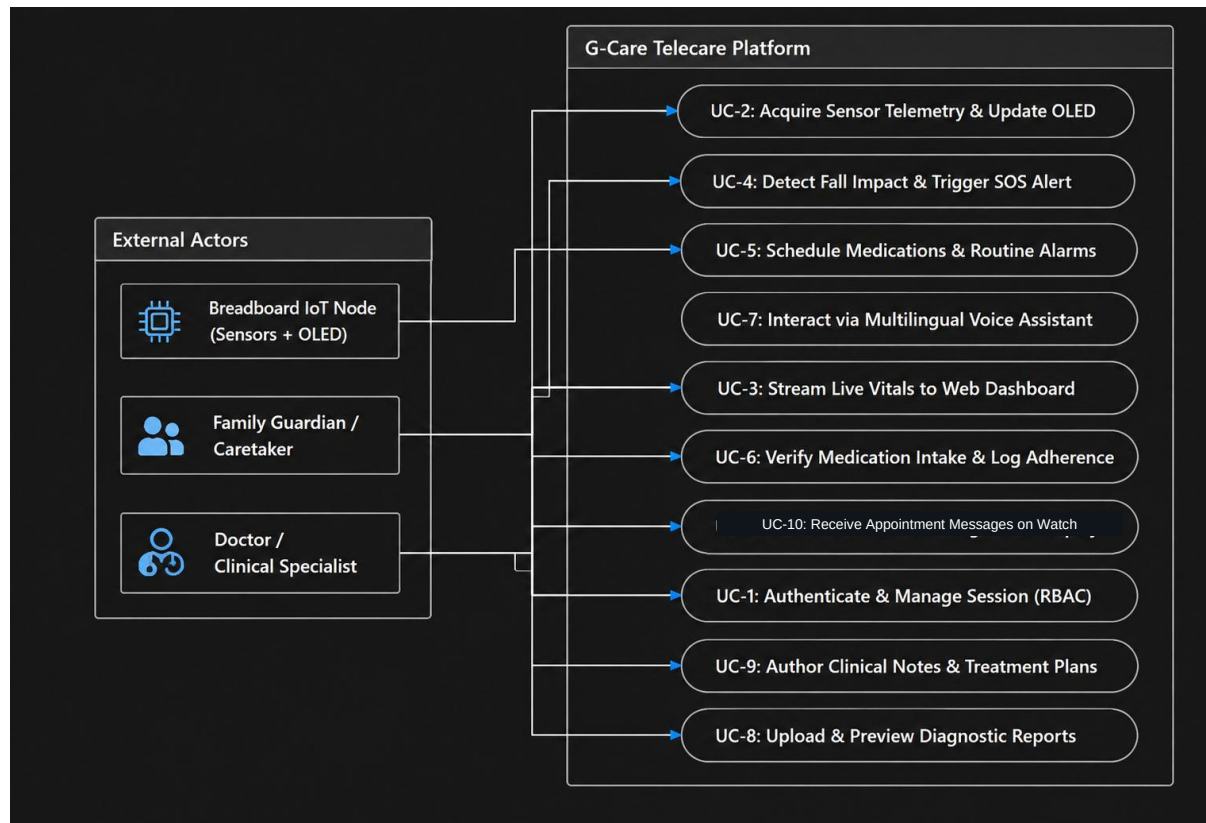


Figure 5.4: Use Case Diagram for the Tri-Tier G-Care System

The diagram's emergency use case refers to application alerts. External ambulance/telecommunications integration and a current GPS hardware flow are outside the implemented scope.

Chapter 6

IMPLEMENTATION

This chapter maps the report to the source that was inspected. Section 6.1 describes hardware/display design and implemented web modules. Section 6.2 separates final software algorithms from illustrative hardware pseudocode. Section 6.3 records authentication, payload validation and storage behavior.

6.1 Implementation of Core Functional Modules

6.1.1 Breadboard Prototype and Sensor Integration

The team reports a physical ESP32-S3 prototype with optical, temperature and inertial modules plus OLED. Controlled movement/shaking triggered the fall/emergency response. The workspace contains web/server code but no deployed .ino/.cpp firmware, measured calibration records or wiring log.

Table 6.1: Breadboard Interfacing and Wiring Design

The original GPIO21/GPIO22 assignment is not an ESP32-S3 mapping. This table retains the wiring functions but marks actual pin assignments for confirmation. Interrupt pins are likewise not asserted as verified connections. The diagram and sample firmware therefore use named, board-specific constants.

Module	Module pin	Board connection	Description / evidence
Common supply	VCC/VIN	Regulated module supply	Original 3.3 V rail design; confirm breakout electrical requirements
Common ground	GND	Common GND	Shared reference
OLED	VCC/GND	Supply/GND	Display power
OLED	SDA	SDA_PIN (confirm)	Shared I2C data
OLED	SCL	SCL_PIN (confirm)	Shared I2C clock; GPIO22 invalid for S3
MAX30102	VIN/GND	Supply/GND	Optical module power
MAX30102	SDA/SCL	SDA_PIN/SCL_PIN	Shared bus; actual mapping not archived
MAX30102	INT	Confirm actual wiring	Original GPIO19 proposal unverified

Table 6.1 (continued): Breadboard Interfacing and Wiring Design

6.1.2 OLED Display Firmware Illustration

The original display example is preserved across this and the next two pages as illustrative C++ pseudocode. It is not a recovered firmware listing: sensor-read helper definitions were absent, and it cannot establish the implemented filtering or refresh timing. Replace board-specific constants with the actual wiring before compilation.

The example retains Wire, Adafruit_GFX and Adafruit_SSD1306, the 128 x 64 dimensions and display address 0x3C. It demonstrates clearing a display buffer and drawing a rectangular layout. Library versions and the deployed sensor drivers still need to be archived.

Module	Module pin	Board connection	Description / evidence
MAX30205	VDD/GND	Supply/GND	Temperature module power
MAX30205	SDA/SCL	SDA_PIN/SCL_PIN	Shared I2C bus; confirm wiring
MPU6050	VCC/GND	Supply/GND	Inertial module power
MPU6050	SDA/SCL	SDA_PIN/SCL_PIN	Shared I2C bus; confirm wiring
MPU6050	INT	Confirm actual wiring	Original GPIO18 proposal unverified

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1
Adafruit_SSD1306
display(SCREEN_WIDTH,
SCREEN_HEIGHT,
&Wire,
OLED_RESET);
void setup() {
  Wire.begin(SDA_PIN, SCL_PIN); // confirm board wiring
  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
  display.clearDisplay();
}
void loop() {
  // Sensor read helpers are integration placeholders
```

The following continues the illustrative display loop. The read helpers represent sensor-driver integration points; their implementations and accuracy are not verified by this listing. A fall flag controls a display status only; physical fall classification is not performed by the drawing calls.

```
int heartRate = readMAX30102HeartRate();    // example value, not a measurement
int spo2Level = readMAX30102SpO2();        // Dynamic SpO2 level (e.g. 97-99
%)
float bodyTemp = readMAX30205Temperature(); // Precision temp (e.g. 36.7 C)
bool fallDetected = checkMPU6050Fall();
// 2. Render Rectangular Dashboard UI
display.clearDisplay();

// Outer frame boundary
display.drawRect(0, 0, 128, 64, SSD1306_WHITE);

// Header section
display.setTextSize(1);
display.setTextColor(SSD1306_WHITE);
display.setCursor(18, 4);
display.print("G-CARE MONITOR");
display.drawLine(0, 14, 127, 14, SSD1306_WHITE);
// Vitals Grid Boxes
// Box 1: Heart Rate
display.setCursor(6, 20);
display.print("HR  : ");
display.print(heartRate);
display.print(" bpm");
// Box 2: SpO2
display.setCursor(6, 32);
display.print("SpO2: ");
display.print(spo2Level);
display.print(" %");
// Box 3: Temperature
display.setCursor(6, 44);
display.print("TEMP: ");
```

Illustrative OLED code (continued):

```
display.print(bodyTemp, 1);
  display.print(" C");
  // Footer Alert Status
  display.drawLine(0, 53, 127, 53, SSD1306_WHITE);
  display.setCursor(6, 55);
  if (fallDetected) {
    display.print("! FALL DETECTED !");
  } else {
    display.print("STATUS: NORMAL");
  }
  // Push buffer to OLED screen
  display.display();
  delay(500); // nominal delay; not a measured refresh rate
}
```

6.1.3 Web Presentation and State Engine

React 18 components use TypeScript models, Tailwind CSS and shadcn/ui. Zustand separates authentication, application state and guardian state. Patient-specific vitals, medications, alarms and alert records use different storage/synchronization paths; a shared interface does not imply identical persistence guarantees.

Recharts renders telemetry/trend views. BroadcastChannel, browser events and localStorage propagate selected changes between same-origin views. The application also polls authoritative alerts and device telemetry on a three-second cycle. Simulated data must be identified when interpreting charts.

6.1.4 Doctor Workspace

DoctorPortal.tsx provides acknowledgement, appointment scheduling, PDF medical records and clinical notes. The sample blood-test upload was demonstrated; keyword-based file validation and category choices are not equivalent to interpreting an ECG or radiology image.

6.1.5 Voice and Alert Audio

VoiceAssistant.ts requests TTS audio from /api/tts and falls back to browser speech synthesis. English, Hindi, Kannada and Tamil messages are implemented. Kannada testing and a Hindi demonstration are the stated physical/browser evidence; automated tests inspect text and speech requests rather than auditory quality.

The current alertAudio implementation loads WAV assets under public/audio and uses browser vibration where supported. It does not implement the earlier report's proposed alternating 880/440 Hz oscillator. Appointment speech uses an end callback to close the confirmation after completion.

6.2 Algorithms and Pseudocode

Algorithm 6.1: Physiological Rules and Stateful Alert Episodes

Algorithm: DetectAndDispatchVitalsAnomalies

Input: elder profile; current heart_rate, spo2, blood pressure, breathing_rate and stress; persisted elder-condition episode records.

Output: created or updated current alert records, synchronized role views and persistence requests.

1. Read the condition lifecycle for the elder. For each metric in its recovery range, mark episode_recovered and rearm the physiological detector. Do not set resolved or guardian acknowledged during recovery.
2. Apply the heart-rate rules: >120 critical; >100 warning; <48 critical; <55 warning. These comparisons are implemented configuration, not validated diagnostic limits.
3. Apply oxygen rules: <90 critical; otherwise <93 warning. Apply BP rules: high critical $\geq 160/100$, high warning $\geq 140/90$, low critical $< 85/55$, low warning $< 90/60$, considering either component.
4. Evaluate breathing rate >26/min and stress >85/100 when applicable. Each detector output includes elder, metric, value, severity, title, recommendation and timestamp.
5. Normalize each output to a canonical type, such as HIGH_HEART_RATE, LOW_HEART_RATE, LOW_SPO2, HIGH_BP or LOW_BP.
6. Form key = elderId + ':' + canonicalType. Values, timestamps and generated record IDs are not part of this identity.
7. Locate an unresolved, unrecovered record for that key. Recovered but still unacknowledged records remain visible for action and are not reused as the new current episode.

The lifecycle describes one continuous condition. The database still stores individual episode record IDs so that a later episode can be distinguished from an earlier alert awaiting review.

Algorithm 6.1 (continued)

8. If a current record exists, update its latest reading/message and severity in place. Synchronize its record ID and state to the application/guardian views. Persist the update without creating another alert.
9. If the episode is acknowledged and awaits recovery, suppress another alert for the same ongoing condition, including a forced tick or severity escalation. Other anomaly types and elders remain independent.
10. If the lifecycle is normal and a new abnormal condition begins, create one alert record, record its episode identity and route its notification. A new generated ID identifies the record but is not the deduplication key.
11. Persist creation/update/acknowledgement/recovery requests in order so quick actions cannot race the initial POST. Reconcile returned server IDs with optimistic browser IDs.
12. The JSON adapter serializes alert read/modify/write operations. The PostgreSQL path locks the elder row inside a transaction. Both apply the stable condition identity and preserve acknowledged state on repeated creation requests.
13. Acknowledge resolves the selected record and silences its episode until recovery. Scheduling an appointment resolves that selected record after booking and broadcasts patient/doctor/date/time/preparation details.
14. On recovery, set `episode_recovered` only. Leave unresolved/acknowledged flags unchanged. Broadcast the condition lifecycle independently of the care workflow.
15. A later abnormal reading after recovery may create a new episode record even if the earlier recovered alert is still awaiting manual review. Acknowledging the older ID must not silence the later episode.
16. Preserve history. Do not delete alerts or truncate records to conceal repeated generation. Reviewed legacy duplicate cleanup is separate from normal telemetry processing. The exported cooldown constant is not the episode identity or permission to regenerate an acknowledged episode. The previous report's 60-second suppression algorithm is replaced by this final lifecycle behavior.

Algorithm 6.2: Inertial Fall Detection Design and Emergency Handling

Evidence boundary: The team reports an implemented multi-stage MPU6050 mechanism and a physical movement-triggered response. The exact firmware is not in this workspace. The following preserves the original algorithm as illustrative design pseudocode, not a verified source listing.

Input: acceleration (A_x , A_y , A_z), angular-rate data and selected timing/threshold parameters.

Output: a candidate fall flag and an application emergency event; not a clinically verified fall classification.

1. Compute acceleration magnitude $A_{mag} = \sqrt{A_x^2 + A_y^2 + A_z^2}$.
2. Start in monitoring state and inspect changes in magnitude over time.
3. In the original design, $A_{mag} < 0.5$ g starts a candidate free-fall window. A subsequent impact ≥ 3.0 g within a proposed 500 ms interval moves to an impact state.
4. Inspect post-impact motion after a proposed 2,000 ms settling interval. The original example uses magnitude near 1 g (within 0.2 g) and angular motion below 15 degrees/s as possible rest indicators.
5. These constants, sensor range settings and state-transition details require confirmation against the deployed firmware. They are not measured performance results and must not be treated as a validated detector specification.
6. If a candidate is confirmed by the firmware, set the fall-detected flag and indicate it locally. Any cancellation countdown or haptic response must be documented and tested separately before it is described as deployed behavior.

The user-reported breadboard test is functional evidence that movement can activate the mechanism. It cannot establish discrimination between human falls and daily activities or quantify sensitivity, specificity and false alarms.

Algorithm 6.2 (continued)

7. The original design includes a 15-second cancellation countdown before escalation. This remains a design parameter because deployed firmware/countdown evidence is unavailable.
8. In the implemented backend, a device packet with `fall_detected=true` is stored and creates a critical fall alert. This is separate from the simulated abnormal-vital episode manager.
9. Web SOS actions create application alerts and synchronize them through the shared notification channel. Automated ambulance calls, SMS and external emergency delivery are not implemented in the audited source.
10. A physical fall-detection validation programme must compare controlled fall-like movement and daily activities using reference labels and recorded sensor traces. No such statistical study is reported here.

Algorithm 6.3: Medication Reminder and Taken Workflow

Input: medicine definitions, timing/frequency fields, patient association and current browser clock.

Output: reminder presentation, interaction state and applicable notification requests.

1. A caretaker or doctor enters the medicine name, dose, instructions and timing through the relevant form. The application stores medicine definitions locally and can submit them to the backend.
2. The watch/browser checks reminders for the selected elder. Due local reminders are marked triggered; guardian alarms are selected using time, verification, dismissal and snooze state.
3. Display the message and request speech in the patient's language. Alert audio/vibration depend on browser capability and permission.
4. Let the user or caretaker mark the reminder Taken/verified or use an available voice confirmation. Do not infer actual ingestion from a button press.

The original proposed 45-minute adherence timer is not an established implementation result. The watch's response overlay and snooze logic must be described using their actual timing rather than conflated with a clinical compliance policy.

Algorithm 6.3 (continued)

5. Update the appropriate reminder state and generate an application notification when the workflow requires it. Guardian reminder verification is an in-memory state mutation; it is not itself a durable per-dose adherence log.
6. Persisted medication definitions and locally stored alarms do not prove persistence of every reminder interaction. Refresh/reload and the interaction between API data and browser state remain validation requirements, particularly given the team's reported disappearing medication information.
7. Treat long-term Taken/Missed history, restart recovery and cross-role reconciliation as future improvements until explicitly verified. Do not present a complete database compliance record as an achieved result.

6.3 Security, API Contracts and Database Connectivity**6.3.1 Role-Based Access and Token Handling**

server/auth.js implements HMAC-SHA256 JWT signing/verification directly using Node crypto. The payload contains sub, role, email, jti, iat and exp. Passwords use a random salt and PBKDF2-SHA512 with 120,000 iterations; bcrypt/jsonwebtoken packages being listed does not mean these libraries implement the active path.

The authenticated server handlers apply role/patient checks. The frontend ProtectedRoute restricts the role portals. /api/auth/logout records a revoked token identifier; session authentication consults that record. This mechanism is implemented, but a post-logout token replay experiment is not claimed.

6.3.2 Payload Validation with Zod

The ordinary /api/vitals handler uses the schema continued on the next page. Its SpO2 field is coerced to an integer; it should not be described as preserving decimal hardware SpO2. Device ingestion follows a different manual parser and is not uniformly covered by this schema.

Vitals Schema (source-aligned excerpt)

The schema uses these coercion/range constraints: `elderId` is a non-empty string; `heart_rate` and `systolic_bp` are integers from 0 to 300; `diastolic_bp` is 0-200; `spo2`, `stress` and `hydration` are integers from 0 to 100; `breathing_rate` is an integer from 0 to 100; `skin_temp` is numeric from 0 to 50. Optional `fall_detected`, `panic_detected` and `shiver_detected` flags default false. `source` is manual, simulator or device; `timestamp` is optional.

These bounds are API contracts, not medical normal ranges. Other handlers use their own checks, and the device route accepts camelCase/snake_case aliases with defaults. It does not currently apply the same complete Zod validation. A claim that all endpoints reject all malformed packets is therefore unsupported.

6.3.3 Alternative Data Persistence Adapters

PostgreSQL: `server/db.js` contains `pg.Pool` setup and schemas for users, elders, role association, medications, alarms, vitals, notes, reports, alerts, audit logs and revoked tokens. Alert creation has transaction/row-lock logic. Automated PostgreSQL tests mock the client; no live PostgreSQL deployment or production database experiment was available.

JSON: Without `DATABASE_URL`, the active local backend uses `data/db.json`. Alerts, medical records and other server entities use file-backed operations. Isolated tests verify alert creation, acknowledgement, recovery flags, reload and role reads, and medical-report creation/retrieval. This is evidence for those paths, not a blanket persistence guarantee for browser-only state.

The adapter is chosen by configuration. PostgreSQL interruption does not imply tested automatic failover or replication into JSON. Operational backups, access controls, concurrent workload testing and stronger medication history remain future deployment work.

Chapter 7

TESTING

Testing is reported by evidence type. The team supplied a history of physical and browser demonstrations. The source audit and current automated run provide additional software evidence. These must not be combined into a claim of clinical validation or uniformly measured hardware/software performance.

7.1 Testing Methodologies and Verification Strategies

The retained testing diagram groups hardware, unit/state, API and workflow checks. Its labels now distinguish reported demonstrations, automated checks and unverified experiments. Physical sensor calibration, measured refresh timing and logout-token replay were not established by the supplied evidence. The report revision used the current workspace on 6 October 2026. Vitest completed 117 tests across 15 files. Many UI/workflow tests use jsdom and mocked APIs/audio; isolated JSON/HTTP tests exercise real local persistence. PostgreSQL tests mock the client. These results apply to the reviewed software revision, not to physical sensor accuracy or browser voice quality.

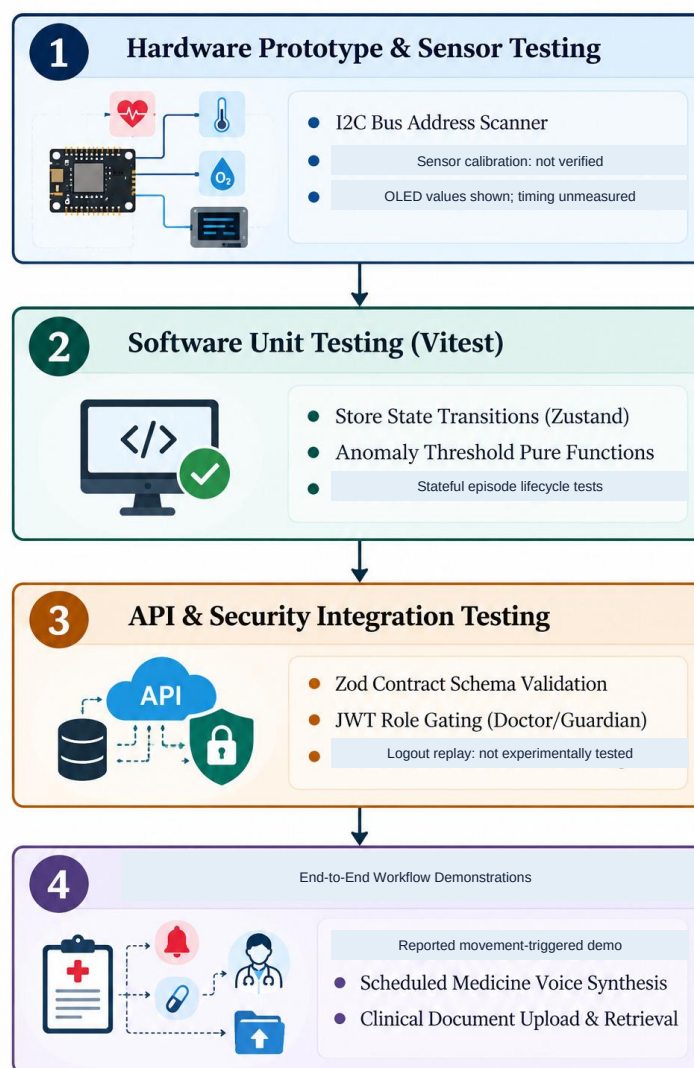


Figure 7.1: Multi-Tier Testing Pipeline and Evidence Boundaries

7.1.1 Hardware Prototype Testing

The team reports a physical breadboard containing the listed modules, OLED telemetry display and movement/shaking that triggers the implemented fall/emergency workflow. These are functional demonstrations on the prototype. No controlled human-fall trial or clinical study was performed.

The original claims of four successful I2C scanner acknowledgements, stable measured 2 Hz refresh, calibrated optical/temperature accuracy and absence of bus faults are not supported by raw logs. They are not marked PASS. Actual bus addresses, firmware and reference-instrument comparisons should be recorded in future tests.

7.1.2 Software Unit and State Testing

The current run passed 117 tests in 15 files. anomalyAlertRouting and vitalsSimulator check configured detection/generation. Episode regression tests cover ten changing low-SpO2 readings, independent elders/types, acknowledgement while abnormal, recovery without automatic resolution, subsequent episodes and reload behavior.

The rendered Watch test checks alert presentation, acknowledgement, scrollable SOS placement, correct-patient appointment messages and Kannada speech requests. speechCompletion tests check audio-end/fallback callbacks and cancellation; playback is mocked, so these do not establish audible voice quality.

7.1.3 Integration, Access and Synchronization

Isolated JSON/HTTP tests verify concurrent alert creation, persisted lifecycle changes and authorized role reads. Medical-report tests check file validation and stored data/role retrieval. Some store tests assert current state rather than a complete browser-restart sequence; their names alone do not prove durable medication compliance.

The team checked doctor/caretaker/guardian separation and guardian patient visibility. It also observed the same HR, SpO2 and temperature across Watch and Guardian interfaces without manual refresh. No numerical latency was recorded, and post-logout token replay was not established.

7.2 Test Cases and Verification Results

Table 7.1 retains all ten original case IDs, with corrected actions, observations and evidence status. PASS-SIM denotes automated simulated/mock verification, DEMO denotes the stated functional demonstration, and NOT VERIFIED identifies missing evidence. A missing experiment is not converted into PASS or FAIL.

Table 7.1: System Test Cases and Evidence-Based Verification Results

Table 7.1: System Test Cases and Evidence-Based Verification Results (TC01-TC06)

ID	Test description	Input / action	Expected behavior	Evidence / actual observation	Status
TC01	I2C discovery	Run scanner on deployed hardware	Configured devices acknowledge	Original addresses retained as design; no scanner log supplied	NOT VERIFIED
TC02	OLED telemetry	Run breadboard display	Changing values/status visible	OLED sensor/simulated display reported; 2 Hz/flicker/memory claims not measured	DEMO
TC03	PPG acquisition /accuracy	Finger/sensor and reference comparison	Acquire optical signal; assess accuracy separately	No reference records establish original 74 bpm/98% or calibration result	NOT VERIFIED
TC04	MPU6050 movement trigger	Controlled breadboard movement/shaking	Detection triggers emergency workflow	Physical trigger reported; no controlled human falls or statistical discrimination	DEMO
TC05	Critical HR anomaly	Inject HR 128 bpm	Critical record and notification request	Configured detector/routing tested with simulated values; audio mocked, no tone measurement	PASS-SIM
TC06	Episode deduplication	Ten changing low-SpO2 ticks; ack, recovery, reload	One continuous episode; manual resolution; new episode after recovery	Regression and isolated JSON/HTTP tests pass; no 60-second recreation	PASS-SIM / JSON

Tests use the final episode lifecycle rather than a time-based alert-generation workaround. Clinical labels in a test message are configured software text, not diagnoses. Physical demonstration evidence does not establish sensitivity or specificity.

Table 7.1 (continued): System Test Cases and Evidence-Based Verification Results (TC07-TC10)

ID	Test description	Input / action	Expected behavior	Evidence / actual observation	Status
TC07	Medication workflow	Caretaker adds medicine; reminder; Taken	Reminder and interaction update	Functional flow reported; complete historical DB adherence remains unverified	DEMO; history limited
TC08	Multilingual voice	Kannada interaction; Hindi demo	Localized speech/messages	Kannada tested; Hindi demonstrated. EN/TA supported, not fully demonstrated	DEMO (KN/HI)
TC09	Medical PDF upload	Upload sample blood-test PDF through doctor	Patient, type, doctor, date and viewing options	Sample upload/role access reported; separate JSON validation/retrieval tests pass	DEMO / JSON
TC10	JWT logout revocation	Logout then replay old token	Old token rejected by revocation check	Code mechanism exists at /api/auth/logout; no replay experiment supplied	NOT VERIFIED

Additional Verified Workflows

Real-time interface demonstration: The team observed matching HR, SpO2 and temperature in the Watch Simulator and Guardian Portal for the same elder without manually refreshing pages. Automated store/synchronization checks support the software path. End-to-end latency was not measured.

Alert lifecycle: Ten consecutive low-SpO2 readings and changing abnormal values remain one ongoing episode. Normal readings mark recovery but leave unacknowledged alerts pending. Acknowledgement while abnormal blocks immediate recreation; later recovery permits a new episode. The regression also keeps all three demo anomalies pending until manual action.

Appointment messages: Booking resolves the selected alert and sends localized patient/date/time/doctor/preparation details. Tests verify message generation, selected-patient delivery, speech requests and completion-based banner dismissal. Browser audio/speech functions are mocked; the automated result is not an audible-language acceptance test.

Production build: The audited revision's Vite production build completed. The existing bundle-size warning remained. A build pass is not a successful production deployment, availability study or medical validation.

Testing limitations: No reference-sensor error analysis, controlled human-fall data, false-alarm statistics, validated recognition accuracy, production PostgreSQL run, quantified user acceptance or long-duration uptime measurement is available.

Chapter 8

CONCLUSION AND FUTURE SCOPE

G-Care implements a student prototype for sharing elder-care information among doctors, caretakers and family guardians. The work combines a physical breadboard demonstration with a browser Watch Simulator and web portals. Its contribution is an implemented care workflow that can be examined and tested; it is not a clinically validated monitoring device.

1. Physical Prototype and OLED Feedback

The ESP32-S3 setup includes MAX30102, MAX30205, MPU6050 and OLED modules. The team reports that sensor or simulated telemetry can be shown on the display. Controlled physical movement triggered the fall/emergency response. Exact firmware, pin mapping and measured calibration data were not archived in the audited workspace.

A multi-stage MPU6050-based fall-detection mechanism was implemented and functionally tested on the physical prototype. Controlled physical movement produced the expected detection and emergency alert response. Controlled human-fall trials, sensitivity/specificity measurement and false-positive/false-negative analysis remain future work.

2. Telemetry and Alert Episodes

The Watch/Guardian demonstration showed matching patient HR, SpO2 and temperature without manual refresh. The browser simulator provides patient-specific baselines and synthetic abnormalities for testing. BP, stress and hydration in this setup are not direct measurements from the listed hardware.

The final alert layer uses one stable elder-condition identity for each continuous abnormal episode. It updates repeated readings in place, preserves manual acknowledgement and re-arms after recovery. Recovery does not resolve a pending care alert. Tests verify these transitions, but they do not establish zero physiological false alarms or reduced alarm fatigue.

3. Medication Workflow

Medication entry, reminder presentation, Taken actions and care notifications were functionally demonstrated. Medicine definitions have storage paths, but complete durable historical compliance remains a limitation. A button/voice acknowledgement does not verify actual administration.

4. Role-Specific Clinical Workspace

Doctor, caretaker and guardian portals provide different views, with authenticated role checks and patient visibility rules. The team checked role separation and guardian-specific access. Signed JWTs, salted PBKDF2 password hashes and token revocation storage are present in code; logout-replay behavior is not claimed as experimentally verified.

A sample blood-test PDF was uploaded and displayed in Medical Records with patient, category, doctor, date and viewing actions. Automated tests verify selected PDF/content checks and JSON record retrieval. General validation of all diagnostic formats and production PostgreSQL storage was not performed.

5. Multilingual Interaction and Appointments

English, Hindi, Kannada and Tamil are supported by message/voice code. Kannada was tested and Hindi demonstrated. Appointment booking sends localized date/time/doctor information to the selected elder's watch. The confirmation closes after speech completes; interrupted playback remains actionable. Automated tests inspect callbacks and mocked speech requests rather than audible language quality.

The software run passed 117 tests across 15 files and the production build completed. These results support selected simulated and persistence workflows. They do not justify claims of high user acceptance, a measured sub-500 ms response, 99.9% uptime, clinical accuracy or a verified monetary saving.

Future Scope

1. Hardware miniaturization and reproducibility: Archive firmware and actual wiring, prepare a custom PCB and evaluate an enclosure, battery, power use and charging. Water resistance, skin-contact safety and medical-device compliance require their own design/testing; they are not current features.
2. Clinical and sensor evaluation: Compare HR, SpO2 and temperature against suitable reference instruments across placement and motion conditions. Conduct controlled fall-like and daily-activity trials with documented labels and quantitative sensitivity, specificity and false-alarm analysis. Clinical use requires a separate validation process.

3. Stronger data history and deployment: Verify medication definitions, per-dose Taken/Missed history and reminder state across refresh, alert actions, disconnect and restart. Evaluate authenticated multi-user workloads, backups and storage recovery. Deploy/test PostgreSQL only when a live instance and operational evidence are available.
4. Predictive and edge analytics: TinyML/TensorFlow Lite, arrhythmia classification and LSTM-based forecasting are possible research directions, not implemented prediction models. They require suitable data, ethical handling, labels, training/evaluation splits and baseline comparisons before accuracy claims.
5. Outdoor connectivity and location: Cellular NB-IoT/LTE-M options such as SIM7080G/SIM7600 and a GPS receiver could support untethered communication. The current hardware has no GPS. Simulated location views must not be treated as validated satellite geofencing.
6. External notifications and interoperability: Consider SMS, IVR or messaging gateways, with delivery/retry and access-control tests. Automated ambulance contact is not current behavior. HL7/FHIR-based exchange with hospital systems would require mapping, consent and interoperability verification.
7. Broader language and accessibility evaluation: Test English and Tamil in addition to Kannada/Hindi, including speech recognition, message comprehension, playback failures and older-adult usability. Measure contrast and touch targets before claiming WCAG conformance or reduced digital-literacy barriers.
8. Performance and mobile support: Measure device-to-dashboard latency, polling behavior, long-duration availability and browser compatibility. Native Android/iOS apps, OS push notifications, biometric login and background synchronization remain proposed extensions. The next stage should emphasize reproducible evidence rather than additional feature claims. A working demonstration is useful, but medical accuracy, reliable unattended operation and clinical effectiveness require different tests.

BIBLIOGRAPHY

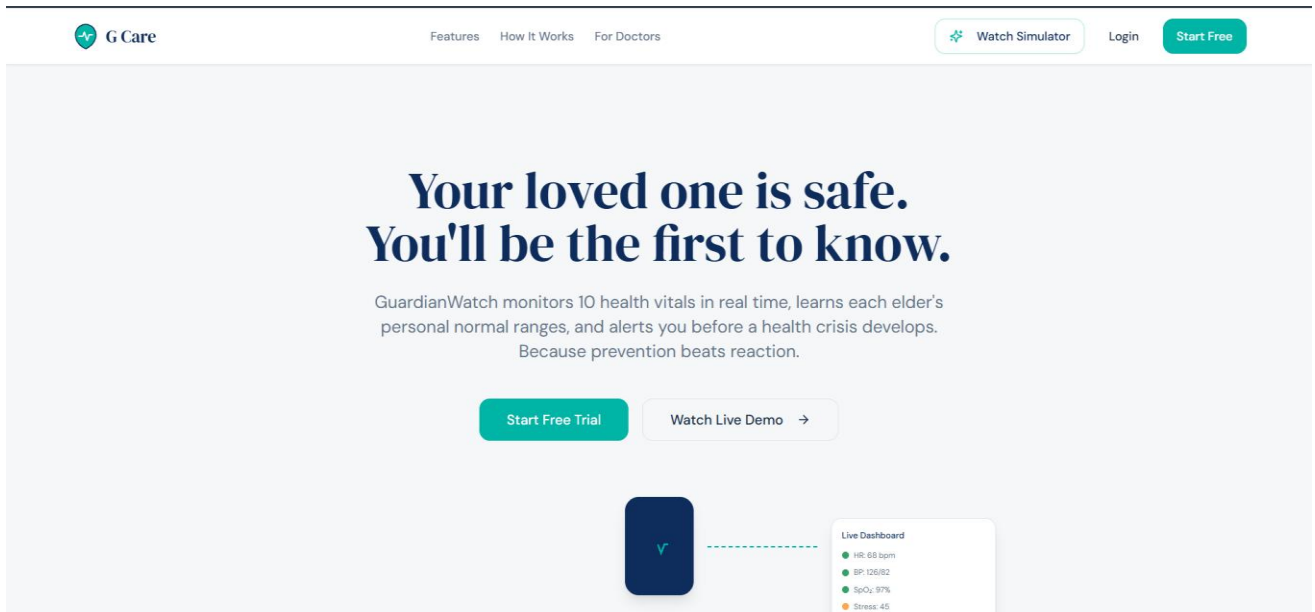
- [1] S. M. R. Islam, D. Kwak, M. H. Kabir, M. Hossain, and K.-S. Kwak, "The Internet of Things for Health Care: A Comprehensive Survey," *IEEE Access*, vol. 3, pp. 678-708, 2015. doi: 10.1109/ACCESS.2015.2437951.
- [2] S. Majumder, T. Mondal, and M. J. Deen, "Wearable Sensors for Remote Health Monitoring," *Sensors*, vol. 17, no. 1, article 130, 2017. doi: 10.3390/s17010130.
- [3] J. Allen, "Photoplethysmography and its application in clinical physiological measurement," *Physiological Measurement*, vol. 28, no. 3, pp. R1-R39, 2007. doi: 10.1088/0967-3334/28/3/R01.
- [4] R. Igual, C. Medrano, and I. Plaza, "Challenges, issues and trends in fall detection systems," *BioMedical Engineering OnLine*, vol. 12, article 66, 2013. doi: 10.1186/1475-925X-12-66.
- [5] N. K. Choudhry et al., "Effect of Reminder Devices on Medication Adherence: The REMIND Randomized Clinical Trial," *JAMA Internal Medicine*, vol. 177, no. 5, pp. 624-631, 2017. doi: 10.1001/jamainternmed.2016.9627.
- [6] P. Nallam, S. Bhandari, J. Sanders, and A. Martin-Hammond, "A Question of Access: Exploring the Perceived Benefits and Barriers of Intelligent Voice Assistants for Improving Access to Consumer Health Resources Among Low-Income Older Adults," *Gerontology and Geriatric Medicine*, vol. 6, article 2333721420985975, 2020. doi: 10.1177/2333721420985975.
- [7] Hardware documentation (corrected from the original combined datasheet description): Espressif Systems, ESP32-S3 Series Datasheet and ESP-IDF GPIO documentation; Analog Devices/Maxim Integrated, MAX30102 and MAX30205 data sheets; TDK InvenSense, MPU-6000/MPU-6050 Product Specification. Documentation consulted 6 October 2026. Component specifications are not prototype validation results.

Hardware	document	locations:
espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf;		
docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-reference/peripherals/gpio.html;		
analog.com/en/products/max30102.html;	analog.com/en/products/max30205.html;	
invensense.tdk.com/products/motion-tracking/6-axis/mpu-6050/.		

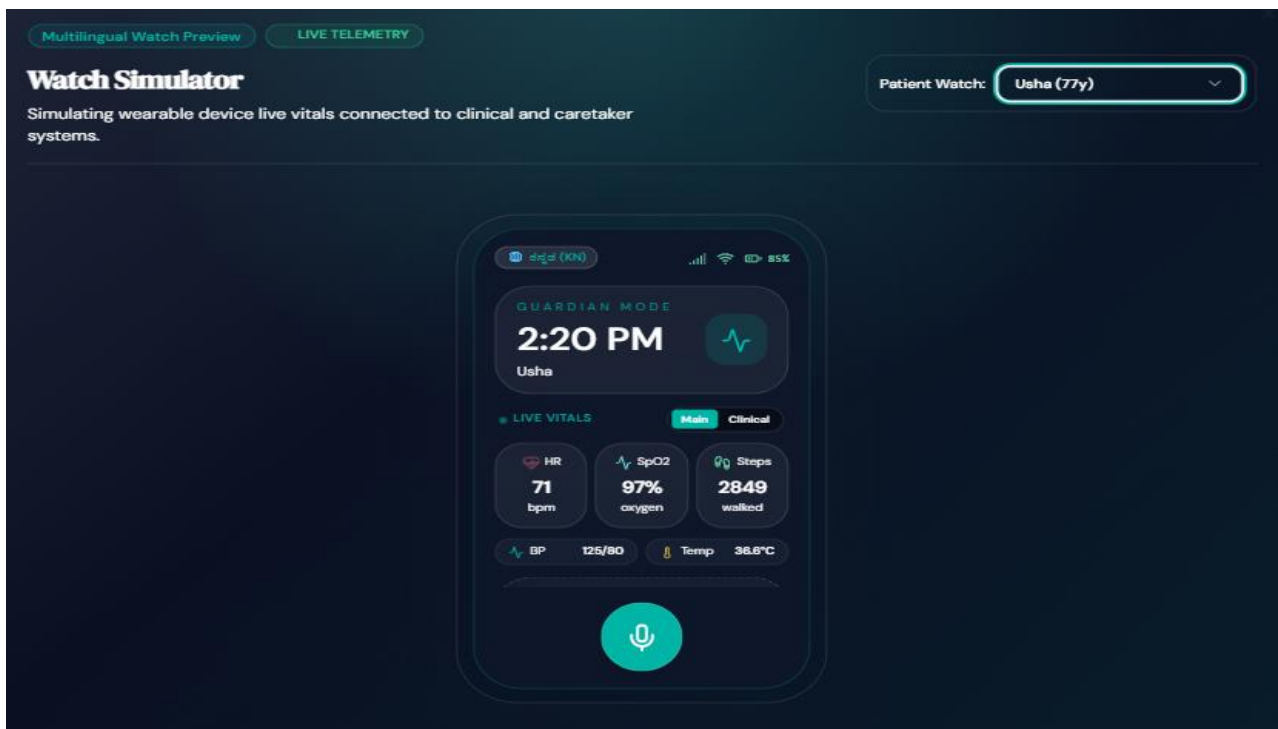
The seven original bibliography entries are retained; citation numbers in Chapter 2 now match them. The corrected [6] title and [7] document attribution do not add fabricated references.

APPENDIX A

SNAPSHOTS



G Care Telecare Platform



G-Care Multilingual Watch Simulator Interface


G Care

Every second matters.

- 10 vitals tracked
- 4 languages
- < 3s alerts
- Guardian Portal

Caretaker
Doctor
Guardian

Welcome back

Email address

Password

☐ Remember me [Forgot password?](#)

Log In

New to GuardianWatch? [Create an account](#)

Multi-Role User Authentication and Login Interface


G Care

Dashboard

Demo Mode
Log out

Y
Yeshrutha S
Caretaker

Dashboard
Elders
Medications
Alarms
Alerts
Reports
Doctor Portal
Settings

AI risk index
24
Weighted from vitals, battery, alerts, and baseline status.

Live devices
3/3
Connected watches streaming recent health signals.

Unresolved alerts
99
Active care events that still need attention.

Stable profiles
2
0 watch needs charging soon.

Highest priority
Watch 57
Venkatesh Rao
Check breathing comfort and keep oxygen trend under review.

SpO2
92.6%

BP
120/77

Stress
61/100

Review vitals
Call family
Locate

AI care queue
Live scoring
Ranked by current risk and monitoring reliability.

VR
Venkatesh Rao
Age 71

HR 81
SPO₂ 92.6%
91%

Check breathing comfort and keep oxygen trend under...
Risk: 57

U
Usha
Age 77

HR 72
SPO₂ 97%
78%

Vitals are inside personal range. Keep routine monitoring active.
Risk: 8

LD
Lakshmi Devi
Age 82

HR 79
SPO₂ 97.1%
54%

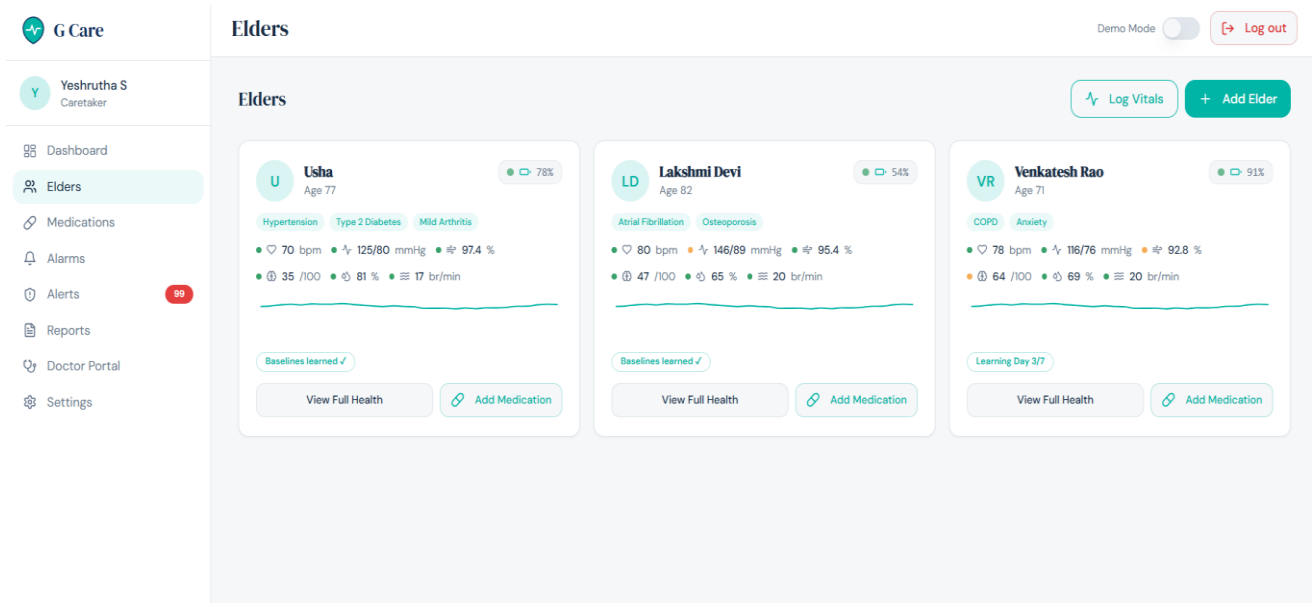
Vitals are inside personal range. Keep routine monitoring...
Risk: 8

Elders

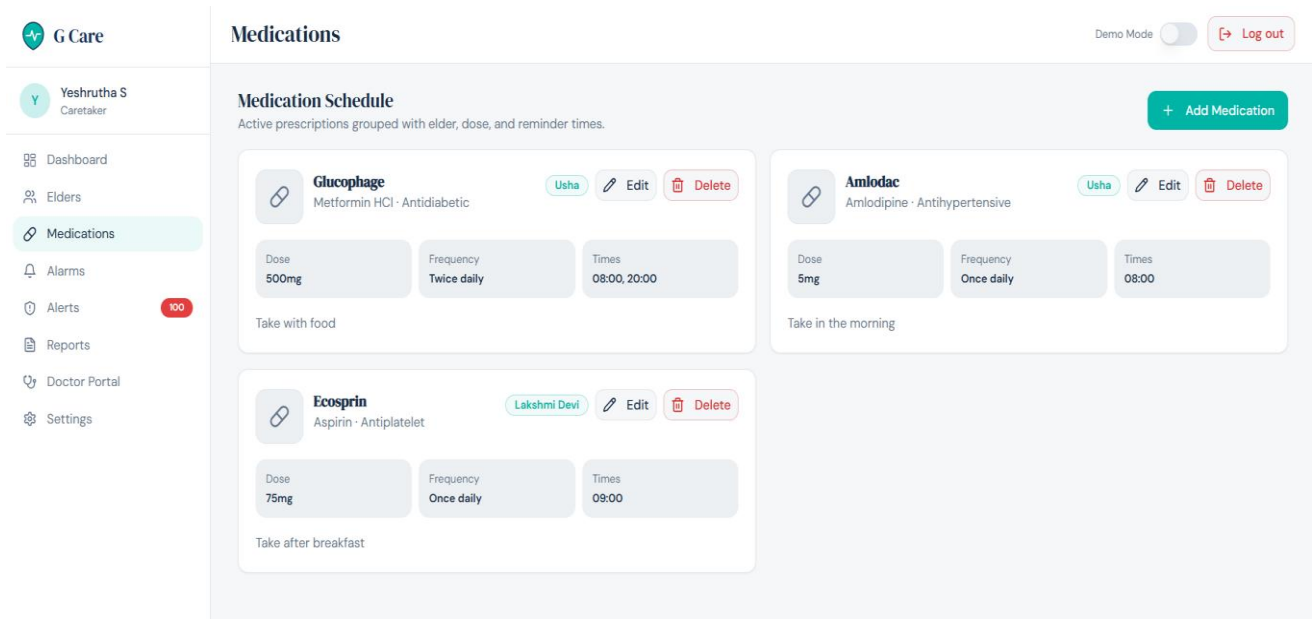
Log Vitals
Add Elder

Caretaker Dashboard for Elder Health Monitoring

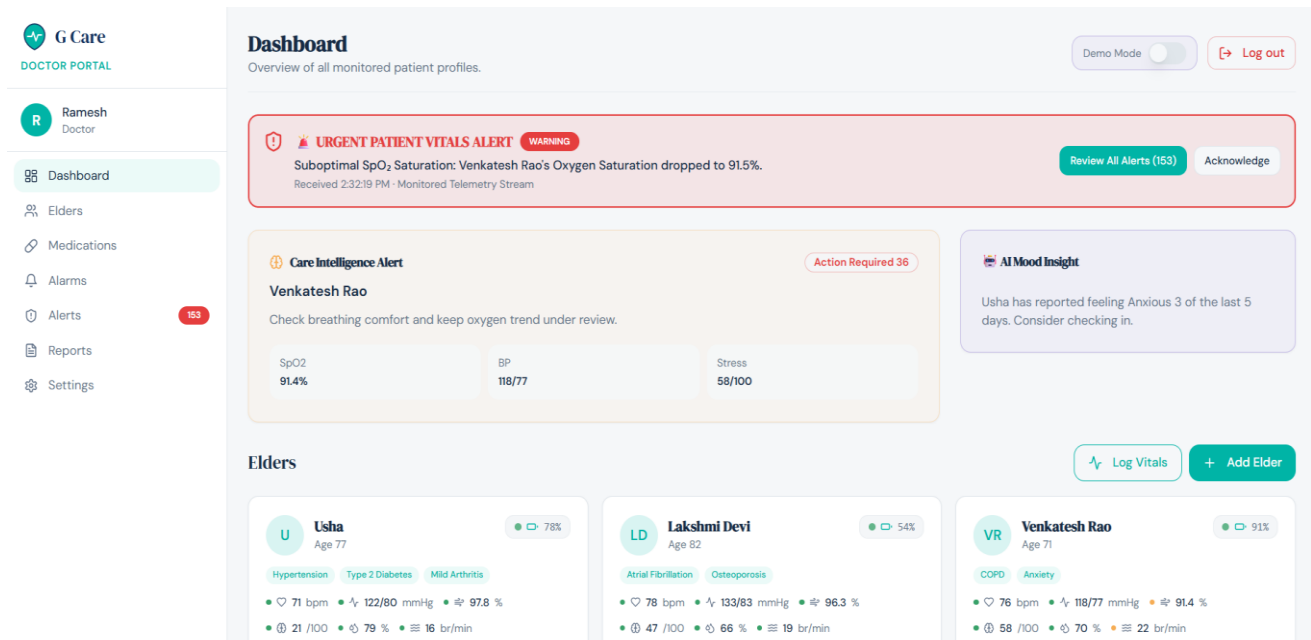
Page 47



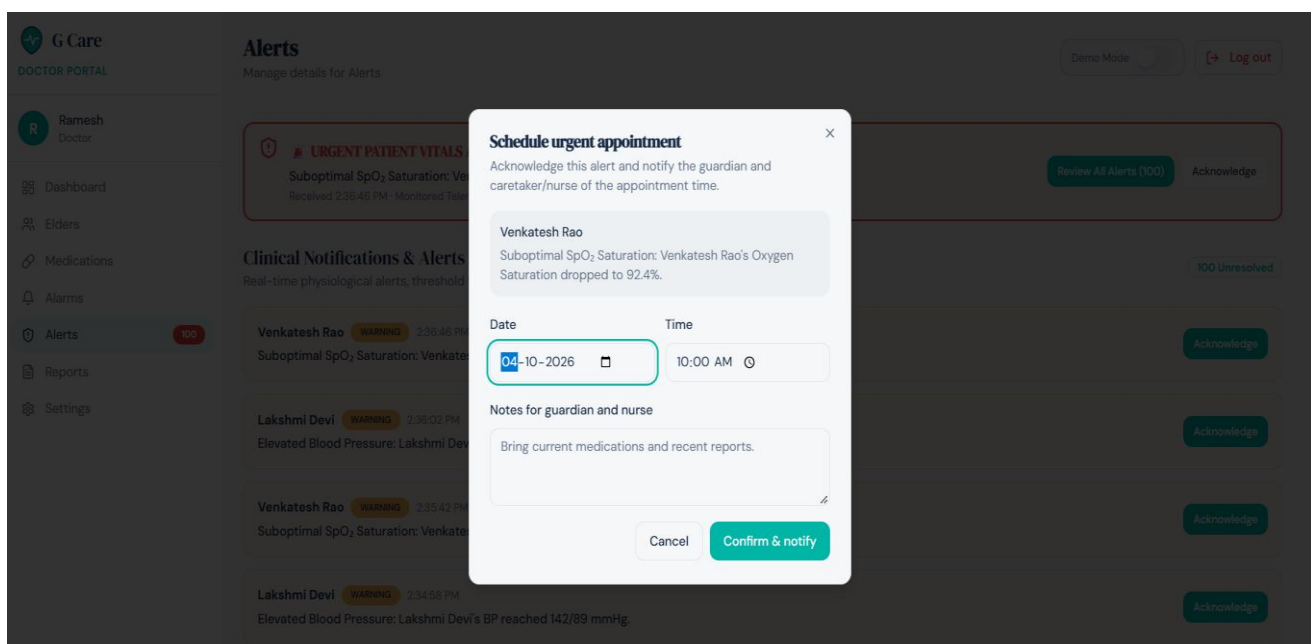
Caretaker Elders Health Monitoring Dashboard



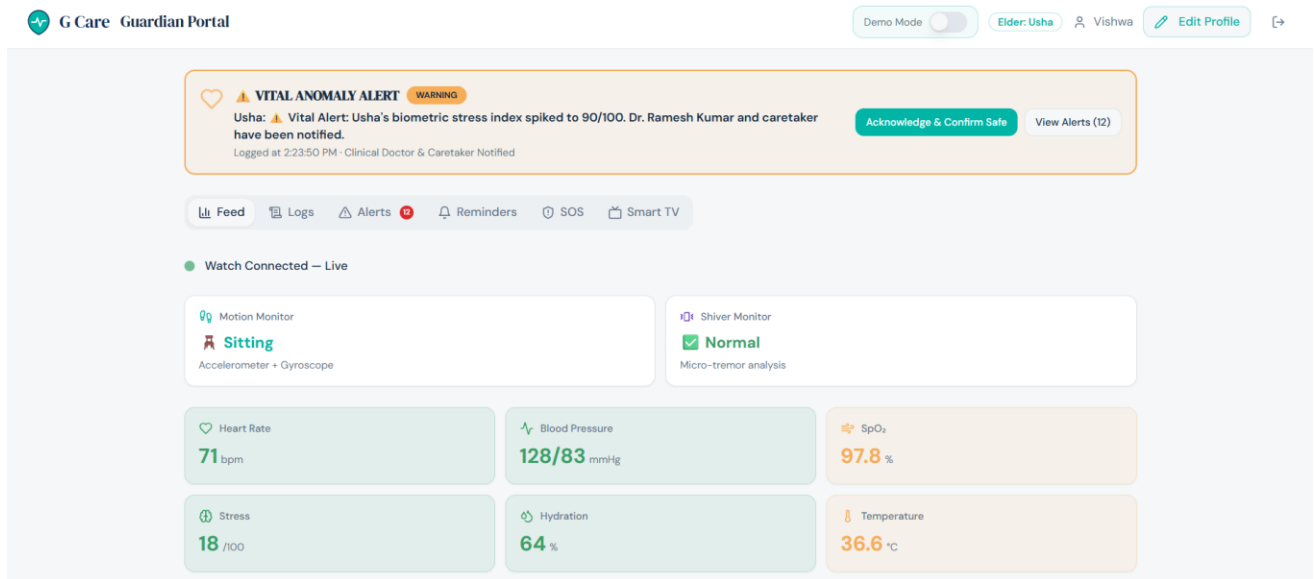
Caretaker Medication Schedule and Management Interface



Doctor Portal Dashboard for Patient Monitoring and Clinical Alerts



Clinical Alert Management and Urgent Appointment Scheduling



Guardian Portal for Real-Time Elder Health Monitoring

The appendix screenshots are retained as historical interface snapshots. They document the original demonstration layout and may predate the final episode lifecycle, acknowledgement changes and scrollable watch controls. They are not evidence of current clinical accuracy or validated adherence history.